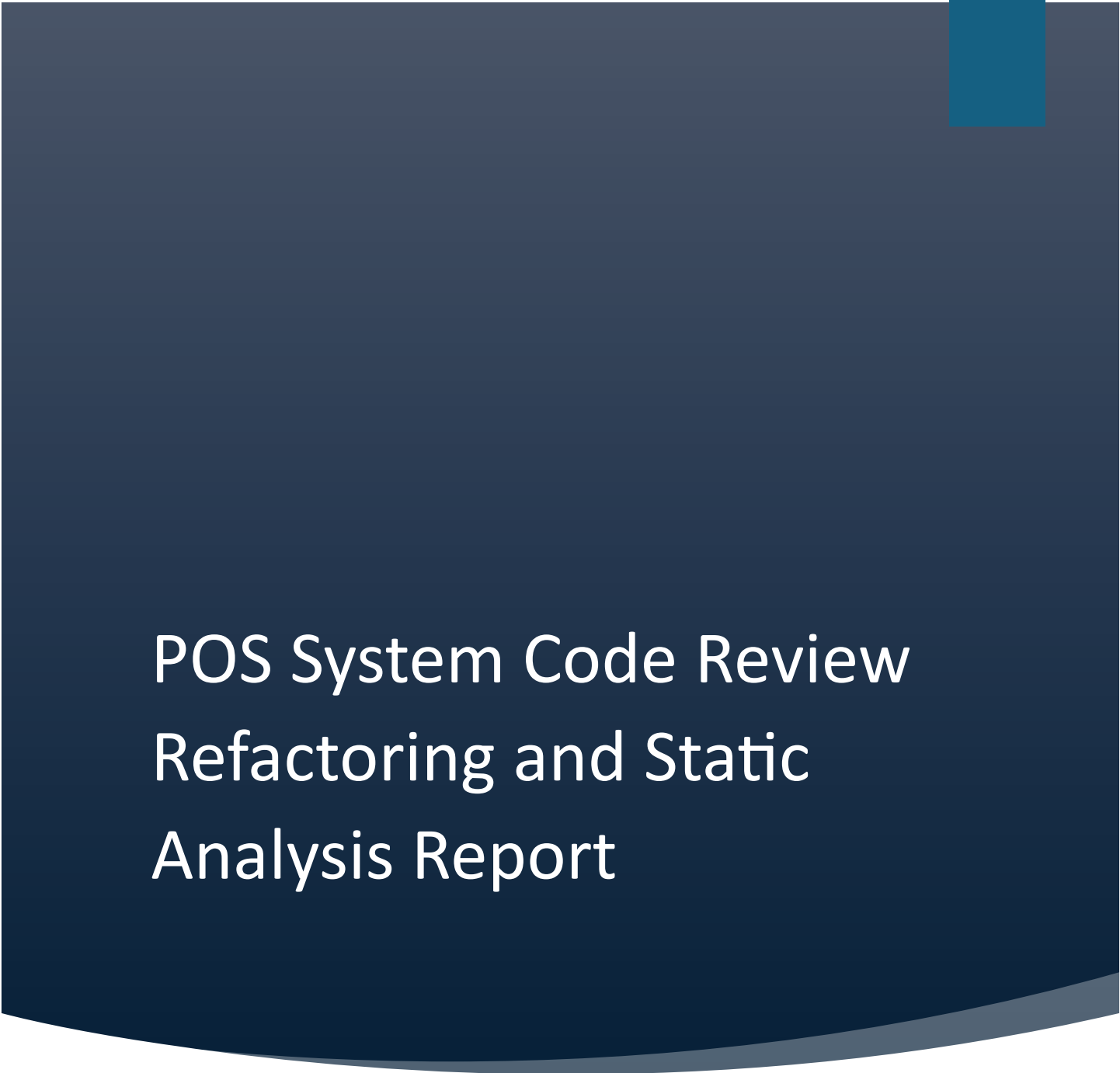




POS System Code Review Refactoring and Static Analysis Report

KASHAF FATIMA

✉ kashaf.fatimaa132@gmail.com

☎ +92 3466208273

Contents

A. Comprehensive Code Review:.....	2
1. AddEmployee_Interface	2
2. Admin_Interface.....	6
3. Cashier_Interface.....	10
4. EnterItem_Interface	13
5. Login_Interface	16
6. Payment_Interface	21
7. Transaction_Interface	24
8. UpdateEmployee_Interface	29
9. Employee.java and its testcases.....	32
10. EmployeeManagement.java and test cases.....	37
11. Inventory.java	44
12. Item.java and test cases	50
13. Management.java and test cases.....	56
14. POH.java and test cases.....	62
15. Pointofsale.java and testcases.....	68
16. POR.java and testcases.....	75
17. POS.java and testcases	82
18. POSSystem.java and testcases.....	89
19. ReturnItem.java and testcases.....	96
20. Register.java and testcases.....	102
B. Refactoring	108
1. POH class	108
2. POSSystems class.....	116
3. POR class.....	126

Introduction

This document presents a thorough Software Quality Assurance (SQA) code review and refactoring analysis conducted on a Java-based Point of Sale (POS) system. Spanning twenty source files including UI interfaces, business logic components, and JUnit test suites, the review systematically evaluates each class against a structured industry-standard checklist covering naming conventions, code structure, exception handling, security, performance, memory management, and maintainability. Where critical issues were identified, hands-on refactoring was performed on three core classes (POH, POSSystem, and POR), supported by detailed before-and-after code comparisons and impact analyses that demonstrate tangible improvements in code quality, modularity, and robustness.

A. Comprehensive Code Review:

1. AddEmployee_Interface

Package Name	Default package
File Name	AddEmployee_Interface.java
Class Name	AddEmployee_Interface

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UID.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).

3	Method Design	Do methods have a single responsibility?	Yes		
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for potential input issues or system initialization.	Add try-catch blocks to handle exceptions such as invalid inputs or system failures.
		Are specific exceptions used instead of generic Exception or Throwable?	N/A		
		Is logging implemented within catch blocks?	No	No logging present for error handling.	Implement logging for exception handling to assist in debugging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	No comments explaining logic, such as cashier registration or admin disposal and reset.	Add comments to clarify the purpose of major sections and complex logic.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on	N/A		

		performance characteristics?			
		Are costly operations minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	POSSystem object is created on each Enter button action, regardless of prior initialization.	Consider initializing POSSystem only if it has not already been initialized elsewhere.
		Are there redundant calculations or excessive logging?	No	Redundant reinitialization of Admin_Interface on every Enter button press.	Initialize Admin_Interface only if needed.
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close	Yes		

		resources in case of exceptions?			
8	Security	Is user input validated to prevent security issues?	No	No validation for user input (e.g., employee name and password fields).	Validate input to prevent issues like SQL injection or unexpected values.
		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	Yes	Duplicated code in handling Admin_Interface re-initialization on the Enter button.	Extract common code into a helper function for re-initializing Admin_Interface.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window size and position (e.g., 520, 200, 500, 280).	Use constants for window size and position values.
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Button and label initialization could be optimized to reduce redundancy and improve readability.	Create a helper method for repetitive UI element initialization to improve modularity.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Repeatedly creating new POSSystem and Admin_Interface instances may	Optimize POSSystem and Admin_Interface initialization by reusing instances if possible.

				lead to memory inefficiency.	
--	--	--	--	------------------------------	--

2. Admin_Interface

Package Name	Default package
File Name	Admin_Interface.java
Class Name	Admin_Interface

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UI D.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	Yes		

		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for potential issues in delete and getEmployeeList.	Add try-catch blocks to handle exceptions such as invalid inputs or database access failures.
		Are specific exceptions used instead of generic Exception or Throwable?	N/A		
		Is logging implemented within catch blocks?	No	No logging present for error handling.	Implement logging for exception handling to assist in debugging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	No comments explaining the logic for UI initialization or employee deletion in removeButton action.	Add comments to clarify the purpose of major sections and complex logic.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on	N/A		

		performance characteristics?			
		Are costly operations minimized inside loops?	No	getEmployeeList() in updateTextArea is recalled every time text area updates, potentially slowing down performance.	Consider caching employeeList if frequently accessed, or calling getEmployeeList only when necessary.
		Is lazy initialization used to defer object creation until it is needed?	Yes		
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to	Yes		

		safely close resources in case of exceptions?			
8	Security	Is user input validated to prevent security issues?	No	Input from JOptionPane for employeeID lacks validation, making it susceptible to invalid data.	Validate user input before processing to prevent potential errors or injection vulnerabilities.
		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	Yes	Duplicated code exists in initializing AddEmployee_Interface and Cashier_Interface based on buttons.	Extract repeated code into helper methods for initializing different interfaces.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window dimensions and button positions, which reduce flexibility.	Use constants for dimensions and positions.
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Duplicated button setup code across different methods for various UI components.	Create helper functions for repetitive UI setup to improve modularity and readability.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Repeated re-initialization of POSSystem and Login_Interface objects can impact memory and efficiency.	Reuse POSSystem and Login_Interface instances instead of creating new ones each time.

3. Cashier_Interface

Package Name	Default package
File Name	Cashier_Interface.java
Class Name	Cashier_Interface

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID is in camelCase.	Rename to SERIAL_VERSION_UID.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	Not provided	Package information missing.	Specify a package name to organize classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	Yes		
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used	N/A		

		properly and not abused?			
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	Yes		
		Are specific exceptions used instead of generic Exception or Throwable?	No	No specific exception handling is used in phone number parsing and validation.	Use NumberFormatException specifically for parsing errors.
		Is logging implemented within catch blocks?	No	No logging present.	Implement logging in exception handling sections for easier debugging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Comments are minimal and don't explain complex logic such as Management handling in unfinished transactions.	Add comments explaining the purpose of each code section.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A		
		Are costly operations	Yes		

		minimized inside loops?			
		Is lazy initialization used to defer object creation?	Yes		
		Are there redundant calculations or excessive logging?	No	Redundant reinitialization of POSSystem occurs.	Initialize POSSystem only once if needed in multiple locations.
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	No	Phone number input is not validated for SQL injection or input sanitization.	Validate and sanitize phone number input.

		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	Yes	Duplicated code exists in handling transaction interface initialization for saleButton, rentalButton, and returnButton.	Extract repeated code into a helper method.
		Are there any magic numbers or hard-coded values?	Yes	9999999999L and 1000000000L in phone number validation are hard-coded.	Replace with named constants (e.g., MAX_PHONE_NUMBER, MIN_PHONE_NUMBER).
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Large button initialization could be moved to a helper function.	Create a method to handle repetitive button setup.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Reinitializing the POSSystem class unnecessarily.	Use a single instance for POSSystem where feasible.

4. EnterItem_Interface

Package Name	Default package
File Name	EnterItem_Interface.java
Class Name	EnterItem_Interface

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UID.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	Yes		
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for potential parsing errors in getItemID and getAmount methods.	Add try-catch blocks for parsing exceptions, especially in getItemID and getAmount methods.
		Are specific exceptions used instead of generic Exception or Throwable?	No	No specific exceptions are used for handling parsing errors.	Use NumberFormatException for parsing errors to handle invalid inputs.
		Is logging implemented within catch blocks?	No	No logging for debugging purposes in case	Implement logging in catch blocks to assist with

				of input parsing failures.	debugging in case of input errors.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments, particularly in actionPerformed where logic could benefit from clarification.	Add comments to describe the purpose and functionality of complex sections.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A		
		Are costly operations minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	Yes		
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		

		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	No	Input for itemID and amount is not validated to prevent non-numeric or excessive input values.	Validate inputs to prevent non-numeric values and ensure valid range.
		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	No		
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window dimensions and component positions, reducing flexibility.	Define constants for dimensions and positions to improve flexibility and readability.
17	Code Smells	Are there code smells not covered by the checklist?	No		
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	No		

5. Login_Interface

Package Name	Default package
File Name	Login_Interface.java
Class Name	Login_Interface

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UID.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	No	actionPerformed method handles multiple button actions, making it perform multiple tasks.	Separate button action logic into distinct methods to improve single responsibility.
		Are method parameters kept to a reasonable limit?	Yes		

		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for input errors or login failures in actionPerformed.	Add try-catch blocks for handling login errors and potential input issues.
		Are specific exceptions used instead of generic Exception or Throwable?	No	Generic exception handling is absent for authentication errors.	Use specific exceptions to capture authentication or login-related issues.
		Is logging implemented within catch blocks?	No	No logging for debugging purposes in case of login failures or incorrect input handling.	Implement logging to capture details in case of exceptions.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments, particularly in actionPerformed, where logic could benefit from clarification.	Add comments to describe the purpose and functionality of complex sections.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on	N/A		

		performance characteristics?			
		Are costly operations minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	POSSystem is initialized within the class scope and used once user credentials are validated, which may lead to unnecessary allocation.	Initialize POSSystem only when needed or consider lazy initialization.
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close	Yes		

		resources in case of exceptions?			
8	Security	Is user input validated to prevent security issues?	No	User credentials are not validated to prevent potential security issues, such as SQL injection or excessive length input.	Validate user inputs to prevent invalid or potentially harmful data.
		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	No		
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window dimensions and component positions, reducing code flexibility and readability.	Define constants for dimensions and positions to improve flexibility and readability.
17	Code Smells	Are there code smells not covered by the checklist?	No		
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	No		

6. Payment_Interface

Package Name	Default package
File Name	Payment_Interface.java
Class Name	Payment_Interface

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UID.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	No	actionPerformed method handles multiple button actions, leading to mixed responsibilities.	Separate button action logic into distinct methods to improve single responsibility.
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for input errors in actionPerformed, especially for	Add try-catch blocks for number format exceptions in PayCash and PayElectronic actions.

				parsing cash and cardNo inputs.	
		Are specific exceptions used instead of generic Exception or Throwable?	No	Missing specific exceptions for number parsing and input validation.	Use NumberFormatException for parsing errors.
		Is logging implemented within catch blocks?	No	No logging implemented for debugging purposes in exception handling.	Implement logging in catch blocks for better debugging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments, particularly in updateText and actionPerformed where logic could benefit from clarification.	Add comments to describe purpose and functionality of complex sections.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A		
		Are costly operations minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	POSSystem and Cashier_Interface are initialized redundantly in multiple action events, such as confirm and cancelTransaction.	Reuse instances of POSSystem and Cashier_Interface where possible to improve efficiency.

		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	No	Input fields for cash and cardNo lack validation, allowing invalid or potentially harmful data.	Validate inputs, especially card number and cash amount, before processing to ensure valid data entry.
		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	Yes	The actionPerformed method is long and handles multiple actions, making it hard to maintain.	Refactor actionPerformed to separate logic for each action, improving readability and maintainability.
		Is there duplicated code that could be refactored?	Yes	Redundant button setup and action logic for transaction cancellation and confirmation.	Extract common code for initializing Cashier_Interface and related actions into helper methods.

		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window size and button positions, reducing code flexibility and readability.	Define constants for dimensions and positions to improve flexibility and readability.
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Repeatedly initializing POSSystem and Cashier_Interface, which can lead to inefficient memory usage.	Use existing instances or initialize once to avoid redundancy and improve performance.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Re-initializing POSSystem and Cashier_Interface in multiple action event cases, impacting efficiency.	Optimize by reusing these instances or initializing only once when needed.

7. Transaction_Interface

Package Name	Default package
File Name	Transaction_Interface.java
Class Name	Transaction_Interface

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase	Yes		

		(e.g., calculateTotalPrice)?			
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UI D.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	No	The actionPerformed method has multiple responsibilities, handling actions for multiple buttons.	Separate button action logic into distinct methods to improve single responsibility.
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for potential input errors in getCustomerPhone method.	Add try-catch blocks for number format exceptions in getCustomerPhone.

		Are specific exceptions used instead of generic Exception or Throwable?	No	Generic exception handling is absent.	Use NumberFormatException in getCustomerPhone.
		Is logging implemented within catch blocks?	No	No logging for exceptions in input handling or transaction processing.	Implement logging to capture details when exceptions occur.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Comments are minimal, especially for logic in actionPerformed and getCustomerPhone .	Add comments to clarify complex logic sections and explain button actions.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A		
		Are costly operations minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	POSSystem and Cashier_Interface objects are repeatedly	Reuse instances of POSSystem and Cashier_Interface where possible.

				initialized in endTransaction and cancelTransaction actions.	
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	No	Input from JOptionPane lacks validation, particularly in getCustomerPhone for phone number inputs.	Validate user input to prevent incorrect values and improve security.

		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	Yes	actionPerformed method is lengthy and handles multiple responsibilities, leading to reduced readability.	Break actionPerformed into separate methods for each button action.
		Is there duplicated code that could be refactored?	Yes	Repeated code for button initialization and handling Cashier_Interface setup in multiple places.	Extract repetitive code into helper functions for better reusability and readability.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window dimensions and positions, as well as phone number range checks.	Use constants for dimensions, positions, and range limits for clarity and maintainability.
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Redundant initialization of POSSystem and Cashier_Interface in multiple methods.	Refactor to reuse objects where possible and reduce redundant code.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Redundant re-initialization of POSSystem and Cashier_Interface on every transaction end or cancel action.	Optimize by reusing these instances if they are already initialized.

8. UpdateEmployee_Interface

Package Name	Default package
File Name	UpdateEmployee_Interface.java
Class Name	UpdateEmployee_Interface

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	serialVersionUID should be in uppercase.	Rename to SERIAL_VERSION_UID.
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	Yes		
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.ui).
3	Method Design	Do methods have a single responsibility?	Yes		
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	No	No exception handling for potential issues in management.update.	Add try-catch blocks to handle exceptions in update operations.
		Are specific exceptions used instead of generic	N/A		

		Exception or Throwable?			
		Is logging implemented within catch blocks?	No	No logging present for error handling.	Implement logging for exception handling to assist in debugging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	No comments explaining the purpose of UI setup and employee update handling in enterButton.	Add comments to clarify the purpose of each code section and complex logic.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A		
		Are costly operations minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	POSSystem and Admin_Interface objects are reinitialized each time the Enter button is pressed, which may be redundant.	Consider reusing instances of POSSystem and Admin_Interface to avoid unnecessary reinitialization.
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A		
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly	Yes		

		closed to prevent resource leaks?			
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	No	Input from JOptionPane for employee details lacks validation, making it susceptible to invalid data entry or injections.	Validate user input before processing to prevent potential errors or security vulnerabilities.
		Are sensitive data encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	Yes	Duplicated code exists in initializing Admin_Interface when the Enter button is pressed.	Extract repeated code into helper methods for re-initializing Admin_Interface.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for window dimensions and component positions, reducing flexibility.	Use constants for dimensions and positions.
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Redundant re-initialization of POSSystem and Admin_Interface objects.	Use existing instances or initialize once if possible.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		

19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Re-initialization of POSSystem and Admin_Interface on every Enter button action affects memory and performance.	Optimize initialization by reusing objects where feasible.
----	----------------------------	--	-----	---	--

9. Employee.java and its testcases

Package Name	Default package
File Name	Employee.java
Class Name	Employee

Checklist Analysis Table

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	N/A	No constants defined in the class.	
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	No	Getter and setter methods lack appropriate access modifiers (should be public).	Add public access modifiers to getter and setter methods.
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing.	Specify a package to organize related classes (e.g., com.pos.models).
3	Method Design	Do methods have a single responsibility?	Yes		

		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A	No method overloading used.	
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	N/A	No exception handling is required in this class.	
		Are specific exceptions used instead of generic Exception or Throwable?	N/A	No exception handling is required in this class.	
		Is logging implemented within catch blocks?	N/A	No exception handling is required in this class.	
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments explaining attributes and methods, which could aid readability.	Add comments to describe the purpose of each attribute and method.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A		

		Are costly operations minimized inside loops?	N/A		
		Is lazy initialization used to defer object creation until it is needed?	Yes	Attributes are only initialized when an Employee object is created.	
		Are there redundant calculations or excessive logging?	N/A		
7	Memory Management	Are unnecessary object references set to null?	N/A	Not applicable for this simple class.	
		Are large objects or collections properly handled to avoid memory leaks?	N/A		
		Are external resources properly closed to prevent resource leaks?	N/A		
		Is try-with-resources used for automatic resource management?	N/A		
		Are large files processed efficiently?	N/A		
		Is RandomAccessFile used appropriately?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	N/A		
8	Security	Is user input validated to prevent security issues?	N/A	No input directly from users in this class.	
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	No	Password is stored as plain text, which poses a security risk if	Use encryption or hashing for storing the password.

				accessed directly.	
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	No		
		Are there any magic numbers or hard-coded values?	N/A	No magic numbers or hard-coded values present.	
17	Code Smells	Are there code smells not covered by the checklist?	No		
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	No		

Checklist Analysis Table for test cases

S#	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Tests only cover basic getters and setters, no null or boundary cases are tested.	Add tests for edge cases, such as null values or empty strings.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	Code coverage is not verified within the test setup.	Implement a code coverage tool (e.g., JaCoCo) to ensure coverage is maintained.

11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes		
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes		
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes		
13	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	No edge case or boundary testing is provided, particularly for invalid or null values.	Add tests to cover edge cases and handle null values.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	No tests for invalid inputs or exceptional scenarios.	Include tests for invalid inputs and ensure error handling is properly tested.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	N/A	No dependencies are present, so mocking is not needed.	
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	N/A	No external dependencies in this class.	
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance tests are implemented, though they	Consider adding performance tests if Employee

				may not be critical for simple getters and setters.	methods are used in high-volume operations.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	Yes		
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	No	No common setup, even though many tests create similar Employee instances.	Use @BeforeEach to initialize common test data to improve maintainability.

10.EmployeeManagement.java and test cases

Package Name	Default package
File Name	EmployeeManagement.java
Class Name	EmployeeManagement

Checklist Analysis Table

S #	Category	Checklist Item	Yes/ No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g.,	No	Constants like employeeDatabase and temp are not in uppercase, which reduces readability.	Rename constants to EMPLOYEE_DATABASE and TEMP for consistency.

		MAX_DISCOU NT)?			
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	No	temp, employeeDatabase, and employees should be private.	Change access modifiers to private for better encapsulation.
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing, which can impact code organization.	Specify a package (e.g., com.pos.management) to organize related classes.
3	Method Design	Do methods have a single responsibility?	No	readFile method does both reading and initializing the list.	Split readFile into two methods: one for reading data and another for initializing the list.
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks?	Yes		
		Are specific exceptions used instead of generic Exception or Throwable?	No	Only FileNotFoundException and IOException are used, which may not be specific enough for some cases.	Add specific handling for other potential exceptions, such as ArrayIndexOutOfBoundsException.
		Is logging implemented	No	Error messages are printed to System.out	Use a logging framework (e.g., java.util.logging) to

		within catch blocks?		without logging for debugging.	provide better debugging information.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments provided, especially in complex methods like update and delete.	Add comments to clarify each method's purpose and key logic.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	Yes	ArrayList is used appropriately for employee storage.	
		Are costly operations minimized inside loops?	No	write operation in the delete method writes to a file within a loop, which may be inefficient.	Write all data to a temporary list or buffer, then write to the file once to reduce IO operations.
		Is lazy initialization used to defer object creation until it is needed?	Yes		
		Are there redundant calculations or excessive logging?	No		

7	Memory Management	Are unnecessary object references set to null?	No	employees list is cleared in readFile, but references are not set to null to aid garbage collection.	Explicitly set references to null where appropriate.
		Are large objects or collections properly handled to avoid memory leaks?	Yes		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	No	try-with-resources is not used, which could prevent efficient resource handling.	Use try-with-resources for file handling to automatically close resources.
		Are large files processed efficiently?	N/A		
		Is RandomAccess File used appropriately?	N/A	Not applicable for this class.	
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	No	User inputs in add, delete, and update are not validated, posing potential security risks.	Validate inputs to prevent issues like invalid or malicious data.
		Are sensitive data (e.g., passwords) encrypted or	No	Passwords are stored as plain text, which is insecure.	Use encryption or hashing for passwords before storing them.

		hashed before storage?			
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	Yes	Code for writing employee data to files is duplicated in add, delete, and update methods.	Refactor file writing logic into a single method to improve maintainability.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded values for file paths like "Database/employeeDatabase.txt" reduce code flexibility.	Use constants for file paths and other values to improve readability and flexibility.
17	Code Smells	Are there code smells not covered by the checklist?	Yes	Multiple System.out.println calls for error handling.	Use a logging framework for error handling instead of System.out.println.
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Repeated file operations within loops can slow down execution, especially for larger datasets.	Consolidate file writes to minimize performance overhead from repeated IO operations.

Checklist Analysis Table for testcases

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes	All public methods (add, delete, update) are covered.	
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Some edge cases are not tested, such as adding employees with null values.	Add tests to cover boundary cases, such as

					null inputs for employee details.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	Code coverage is not verified within the test setup.	Implement a code coverage tool (e.g., JaCoCo) to ensure coverage is maintained.
1 1	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes	All tests follow the Arrange, Act, Assert pattern.	
		Are individual test cases independent of each other (no shared state between tests)?	Yes	Each test initializes and clears its test data independently, ensuring no shared state.	
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes	Test method names are descriptive and indicate their purpose, such as testAdd_Cashier and testUpdate_ValidUpdate.	
1 2	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes	Assertions such as assertEquals, assertTrue, and assertNotEquals are used appropriately.	
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes	Specific assertions are used, such as assertEquals for values.	
1 3	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	Boundary and edge cases, such as null values and maximum size inputs, are not tested.	Add test cases for null inputs and unusual boundary values.

		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling <code>NullPointerException</code>)?	Yes	Tests cover invalid inputs such as <code>nonExistentUsername</code> in <code>testDelete_NonExistingEmployee</code> .	
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	Direct file operations are used without mocking, which may slow down tests and create dependencies.	Use Mockito or other mocking frameworks to mock file operations for isolation.
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	File operations are not mocked, which may impact test performance and independence.	Mock file operations to avoid dependence on actual file system resources.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance tests are implemented, though they may not be critical for this class.	Consider adding performance tests for file operations if they are used frequently in production.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	Yes	Tests are modular and avoid duplication by using <code>@After</code> for cleanup.	
		Is there a setup method (<code>@BeforeEach</code>) for initializing common objects and resources used across tests?	Yes	The <code>@Before</code> method sets up the test object and resources needed by each test.	

11.Inventory.java

Package Name	Default package
File Name	Inventory.java
Class Name	Inventory

Checklist Analysis Table

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	N/A	No constants are defined in the class.	
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	No	uniqueInstance should be private according to the Singleton pattern.	Change uniqueInstance to private.
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing, which affects code organization.	Specify a package (e.g., com.pos.inventory) to organize related classes.

3	Method Design	Do methods have a single responsibility?	Yes		
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A	No method overloading is used.	
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	No	No handling inside IOException catch in updateInventory method; the catch block is empty.	Add meaningful exception handling or logging in the catch block.
		Are specific exceptions used instead of generic Exception or Throwable?	Yes	Specific exceptions like FileNotFoundException and IOException are used.	
		Is logging implemented within catch blocks?	No	No logging is used for debugging or tracking errors.	Use a logging framework (e.g., java.util.logging) to provide better error tracking and debugging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments explaining methods and logic, especially in updateInventory and accessInventory.	Add comments to clarify each method's purpose and key logic.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for	Yes		

		variables, classes, and methods?			
6	Performance	Are data structures chosen based on performance characteristics?	Yes	ArrayList is used appropriately for storing items.	
		Are costly operations minimized inside loops?	No	updateInventory has nested loops for each transaction item, which could affect performance with large lists.	Optimize the loop or consider alternative data structures for faster item lookups.
		Is lazy initialization used to defer object creation until it is needed?	Yes	Singleton instance uniqueInstance is lazily initialized.	
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null?	N/A	No unnecessary object references are present.	
		Are large objects or collections properly handled to avoid memory leaks?	Yes		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is try-with-resources used for automatic resource management?	No	try-with-resources is not used, which could prevent efficient resource handling.	Use try-with-resources for file handling to automatically close resources.
		Are large files processed efficiently?	N/A		

		Is RandomAccessFile used appropriately?	N/A	Not applicable for this class.	
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues?	N/A	No direct user input handling occurs in this class.	
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	Not applicable as no sensitive data is managed in this class.	
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	No		
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded file paths like "Database/itemDatabase.txt" reduce flexibility.	Use constants for file paths to improve readability and maintainability.
17	Code Smells	Are there code smells not covered by the checklist?	No		
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not covered by the checklist?	Yes	Nested loops in updateInventory could be optimized for better performance.	Refactor loops to reduce complexity, potentially using Map for faster lookups.

Checklist Analysis Table for testcases

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes	All public methods (getInstance, accessInventory, updateInventory) are covered.	
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Edge cases, such as null inputs for the inventory list or empty transaction items, are not covered.	Add tests for null and boundary values for the inventory and transaction lists.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	Code coverage is not measured within the setup or verified through a tool.	Implement a code coverage tool (e.g., JaCoCo) to maintain an acceptable level of test coverage.
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes	All tests follow the Arrange, Act, Assert pattern.	
		Are individual test cases independent of each other (no shared state between tests)?	Yes	Tests are independent; each test uses a fresh setup.	
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes	Test method names clearly describe the intended functionality, e.g., testUpdateInventoryReturn_PotatoChips.	

1 2	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes	Assertions like assertEquals, assertTrue, and assertSame are used correctly.	
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes	Specific assertions, such as assertEquals, are used appropriately.	
1 3	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	Boundary cases, such as empty or null lists for inventory and transactions, are not tested.	Add test cases to cover empty and null inputs for inventory and transactions.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	Invalid inputs are not tested, and exception handling scenarios (e.g., null database file path) are missing.	Add test cases to handle invalid inputs and exception scenarios.
1 4	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	The test depends directly on file operations, which could affect test reliability.	Use mocks or stubs (e.g., Mockito) for file operations to improve isolation and reduce external dependency.

		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	File operations are not mocked, leading to dependencies on actual files.	Mock file operations to avoid dependency on external file system resources.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance tests are implemented, though they may not be critical for this class.	Consider adding performance tests for large inventory and transaction lists if scalability is a concern.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	Yes	Test methods are modular and avoid duplication through helper methods like createTempDatabase.	
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	Yes	The @Before method initializes necessary objects and resources for each test case.	

12.Item.java and test cases

Package Name	Default package
File Name	Item.java
Class Name	Item

Checklist Analysis Table

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		

		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	N/A	No constants are defined in the class.	
2	Code Structure	Are access modifiers (public, private, protected) used correctly?	No	Some methods (getItemName, getItemID, getPrice, getAmount) do not specify public access, limiting accessibility outside the package.	Add public access modifier to all getter methods to allow access outside the package.
		Are classes and interfaces clearly separated?	Yes		
		Are packages used appropriately to organize related classes?	No	Package information is missing, which affects code organization.	Specify a package (e.g., com.pos.model).
3	Method Design	Do methods have a single responsibility?	Yes		
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly and not abused?	N/A	No method overloading is present in this class.	
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	N/A	No exception handling is required in this simple data model class.	
		Are specific exceptions used instead of generic Exception or Throwable?	N/A	Not applicable as there is no exception handling required.	

		Is logging implemented within catch blocks?	N/A		
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Comments are minimal and could be more descriptive, especially around constructor and key methods.	Add comments explaining the purpose of each attribute and method.
		Is there consistent indentation?	Yes		
		Are blank lines used appropriately?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on performance characteristics?	N/A	Not applicable for this class as it is a simple data model.	
		Are costly operations minimized inside loops?	N/A	Not applicable for this class.	
		Is lazy initialization used to defer object creation until it is needed?	N/A	No lazy initialization is necessary for this class.	
		Are there redundant calculations or excessive logging?	N/A		
7	Memory Management	Are unnecessary object references set to null?	N/A	No unnecessary object references are present.	
		Are large objects or collections properly handled to avoid memory leaks?	N/A	Not applicable for this class.	

		Are external resources properly closed to prevent resource leaks?	N/A	Not applicable as this class does not handle external resources.	
		Is try-with-resources used for automatic resource management?	N/A	No resource management required in this class.	
		Are large files processed efficiently?	N/A	Not applicable for this class.	
		Is RandomAccessFile used appropriately?	N/A	Not applicable for this class.	
		Is error handling implemented to safely close resources in case of exceptions?	N/A		
8	Security	Is user input validated to prevent security issues?	N/A	No direct user input is handled in this class.	
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	Not applicable as no sensitive data is managed in this class.	
9	Maintainability	Are there long methods or deeply nested loops?	No	Methods are short and straightforward.	
		Is there duplicated code that could be refactored?	No		
		Are there any magic numbers or hard-coded values?	No	No magic numbers are present.	
17	Code Smells	Are there code smells not covered by the checklist?	No		
18	Coding Standards	Are there coding standard violations not covered by the checklist?	No		
19	Performance Inefficiencies	Are there performance inefficiencies not	N/A		

		covered by the checklist?			
--	--	---------------------------	--	--	--

Package Name	Not specified in the provided code
File Name	ItemTest.java
Class Name	ItemTest

Checklist Analysis Table

S#	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values)?	No	Tests do not check edge cases, such as negative values or zero values for amount and price.	Add edge cases to test with values such as 0, negative prices, and very high values.
		Is code coverage measured and maintained at an acceptable level (e.g., 80%)?	No	Coverage tools (e.g., Jacoco) are not mentioned or implemented.	Use code coverage tools to measure and ensure an acceptable coverage level.
11	Test Design	Are tests written to follow the AAA pattern for clarity and consistency?	Yes		
		Are individual test cases independent of each other?	Yes		
		Are meaningful and descriptive names used for test methods?	No	Method names like testItemName are generic and could be more	Rename methods to indicate exact behavior being tested, e.g., testGetItemNameReturnsName.

				descriptive to indicate test behavior.	
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue)?	Yes		
		Are specific assertions used instead of general ones?	Yes		
13	Boundary and Edge Cases	Are edge cases and boundary conditions tested?	No	Edge cases like zero, negative, and extreme values are not tested.	Add tests to cover these cases.
		Are invalid inputs and exceptional scenarios covered by tests?	No	Exceptional scenarios (e.g., null or invalid inputs) are not tested.	Add tests to handle such scenarios and verify their handling in methods if applicable.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test?	N/A	Not required for this simple data model class.	
		Are dependencies mocked to avoid external calls?	N/A	No external dependencies are present.	
15	Performance Testing	Are there tests to check performance for critical methods?	N/A	Not applicable for this simple data model class without performance-critical methods.	
16	Test Maintainability	Are test methods well-organized and modular, avoiding	Yes		

		duplication of code?			
		Is there a setup method (@BeforeEach) for initializing common objects?	Yes		

13.Management.java and test cases

Package Name	Default package
File Name	Management.java
Class Name	Management

Checklist Analysis Table

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase?	Yes		
		Are variable and method names written in camelCase?	Yes		
		Are constants written in uppercase with underscores?	Yes	Constants are not defined in uppercase for consistency.	Rename userDatabase to USER_DATABASE.
2	Code Structure	Are access modifiers used correctly according to Java standards?	Yes		
		Are classes and interfaces clearly separated?	Yes		

		Are packages used appropriately to organize related classes?	No	No package is defined.	Define an appropriate package (e.g., com.pos.management).
3	Method Design	Do methods have a single responsibility?	No	Some methods, such as checkUser, handle multiple responsibilities, making them complex.	Refactor complex methods into separate, single-responsibility methods.
		Are method parameters kept to a reasonable limit?	Yes		
		Is method overloading used properly?	Yes		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	No	Exception handling is basic, with print statements instead of logs or user feedback.	Implement proper logging or user feedback in catch blocks.
		Are specific exceptions used instead of generic Exception or Throwable?	Yes		
		Is logging implemented within catch blocks for debugging purposes?	No	Logging is absent in catch blocks, with only print statements for exception messages.	Use a logging framework (e.g., Log4j or Java Util Logging).
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Complex logic lacks sufficient comments, making the code harder to follow.	Add comments to clarify steps, particularly within complex methods like getLatestReturnDate.
		Is there consistent indentation (4	Yes		

		spaces or a tab size of 4)?			
		Are blank lines used appropriately to separate code blocks?	Yes		
		Is the code easy to read and understand?	No	The code is readable, but the lack of comments makes it harder to understand at a glance.	Add comments and refactor complex methods for readability.
		Are meaningful names used for variables, classes, and methods?	No	Some variable names like b and nextPh are not descriptive enough.	Use more descriptive variable names (e.g., isReturned, nextPhoneNumber).
6	Performance	Are data structures chosen based on their performance characteristics?	Yes		
		Are costly operations minimized inside loops?	No	Some operations, like creating new objects within loops, could be optimized.	Move object creation outside loops where possible.
		Is lazy initialization used to defer object creation until needed?	Yes		
		Are there redundant calculations or excessive logging?	No	No logging framework is used, and no redundant calculations were noted.	
7	Memory Management	Are unnecessary object references set to null to aid	No	Unused references are not set to null when no longer needed.	Set unused references to null to aid garbage collection, especially with large objects.

		garbage collection?			
		Are large objects or collections properly handled to avoid memory leaks?	Yes		
		Are external resources properly closed to prevent resource leaks?	Yes		
		Is the try-with-resources statement used for automatic resource management?	No	try-with-resources is not used for BufferedReader, FileReader, and similar objects, which are closed manually.	Use try-with-resources for better resource management.
		Are large files processed efficiently?	Yes		
8	Security	Is user input validated to prevent security issues?	No	No input validation is performed, which may expose the system to potential risks.	Add input validation, especially in methods where user data is processed (e.g., checkUser).
		Are sensitive data encrypted or hashed before storage?	No	Sensitive information, such as user data, is not encrypted.	Encrypt sensitive data where applicable.
9	Maintainability	Are there long methods or deeply nested loops?	Yes	Some methods are lengthy with deep nesting, particularly getLatestReturnDate.	Refactor to reduce complexity and improve readability.
		Is there duplicated code that could be refactored?	Yes	Duplicate code blocks, such as exception handling and file reading, are repeated.	Create helper methods to reduce redundancy.
		Are there any magic numbers	Yes	Hard-coded values, such as file paths and	Use constants for these values to enhance

		or hard-coded values?		date formats, reduce flexibility.	flexibility and maintainability.
17	House Keeping - Code Smells	Are there any code smells not covered by the checklist?	Yes	Code is cluttered with unused comments and redundant print statements.	Remove unused comments and replace print statements with logs.
18	House Keeping - Coding Standards	Are there any violations of coding standards not covered by the checklist?	Yes	The code could benefit from method documentation comments for better code maintainability.	Add Javadoc comments to methods to clarify their functionality and usage.
19	House Keeping - Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist?	Yes	The addRental and updateRentalStatus methods read and write the entire database, which may be inefficient for large files.	Consider using a database management system or improve file handling for better performance.

Checklist Analysis Table for testcases

S#	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Some boundary cases (e.g., invalid file paths or corrupt database entries) are not covered.	Add tests for boundary cases and invalid input handling.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	Yes		
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert)	Yes		

		for clarity and consistency?			
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods?	Yes		
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes		
13	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions tested?	No	Some edge cases, such as empty rental lists or invalid phone numbers, are not covered.	Add additional tests for edge cases.
		Are invalid inputs and exceptional scenarios covered by tests?	No	Limited testing for exceptional cases; invalid inputs like empty files or null values are not tested.	Implement additional tests to cover exceptions and invalid inputs.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	No use of mocking frameworks; tests directly interact with actual files instead of mocks.	Use mocking frameworks like Mockito to isolate file interactions.
		Are dependencies mocked to avoid external calls?	No	Direct interactions with files could lead to slow tests and dependency on external factors.	Mock file interactions to improve test speed and isolation.

15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	Performance tests are absent; could be beneficial for large file interactions or long-running operations.	Add performance tests for file operations and large data handling.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	No	Repeated code for file operations, e.g., creating temp files and checking rentals.	Refactor repeated code into helper methods or use annotations like <code>@BeforeEach</code> .
		Is there a setup method (<code>@BeforeEach</code>) for initializing common objects and resources?	No	<code>@Before</code> is used, but some setup and cleanup tasks could be moved to improve readability.	Consider consolidating setup code into <code>@BeforeEach</code> and moving cleanup code to <code>@AfterEach</code> .

14.POH.java and test cases

Package Name	Default package
File Name	POH.java
Class Name	POH

Checklist Analysis Table

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with	N/A	No constants present in the code.	

		underscores (e.g., MAX_DISCOUNT)?			
2	Code Structure	Are access modifiers used correctly according to Java standards?	No	Some variables and methods lack access modifiers, e.g., returnList, transactionItem.	Use appropriate access modifiers to ensure encapsulation.
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes		
		Are packages used appropriately to organize related classes?	No	Package not specified.	Define a package to organize this class, e.g., com.pos.inventory.
3	Method Design	Do methods have a single responsibility?	No	Methods such as retrieveTemp handle multiple responsibilities (e.g., reading and updating state).	Split methods to adhere to the single-responsibility principle.
		Are method parameters kept to a reasonable limit (preferably fewer than 5)?	Yes		
		Is method overloading used properly and not abused?	Yes		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	Yes		
		Are specific exceptions used instead of generic Exception or Throwable?	Yes		

		Is logging implemented within catch blocks for debugging purposes?	No	No logging framework used; System.out.println used instead.	Replace System.out.println with a logging framework such as Log4j or java.util.logging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Minimal comments provided, making code harder to follow.	Add comments for complex logic and unclear sections.
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes		
		Are blank lines used appropriately to separate code blocks for better readability?	Yes		
		Is the code easy to read and understand?	No	Use of unclear variable names like t and limited comments affects readability.	Use more descriptive variable names and add inline comments.
		Are meaningful names used for variables, classes, and methods?	No	Variable t and method retrieveTemp are vague and could be clearer.	Use descriptive names like returnSaleFile and retrieveTemporaryData.
6	Performance	Are data structures chosen based on their performance characteristics?	Yes		
		Are costly operations (e.g., database calls) minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	Yes		
		Are there redundant calculations or excessive logging?	Yes	System.out.println statements are scattered	Remove unnecessary println statements or replace them with

				throughout, impacting performance in production.	logging at appropriate levels.
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	No	Temporary files and streams are not explicitly set to null after use, which may hinder GC.	Set unused objects and references to null to encourage garbage collection.
		Are large objects or collections properly handled to avoid memory leaks?	Yes		
		Are external resources properly closed to prevent resource leaks?	No	While streams are closed, some resource closing (e.g., in <code>updateRentalStatus</code>) could be more robust.	Use try-with-resources where possible for automatic resource management.
		Is the try-with-resources statement used for automatic resource management?	No	Standard try-catch is used for file handling instead of try-with-resources.	Use try-with-resources for automatic resource closing.
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	N/A	User input validation is not required in this code segment.	
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	Not applicable in this class.	
9	Maintainability	Are there long methods or deeply nested loops?	Yes	The <code>endPOS</code> method is lengthy and contains nested loops, making it hard to follow and maintain.	Refactor <code>endPOS</code> into smaller methods.

		Is there duplicated code that could be refactored?	Yes	Code for file reading and writing is duplicated in multiple methods.	Extract repeated file operations into a separate helper method.
		Are there any magic numbers or hard-coded values?	Yes	Hardcoded values, such as file paths and 0.1, make the code less flexible.	Define constants for file paths and percentage values.
17	Code Smells	Are there any code smells not covered by the checklist that are present?	Yes	Long methods, unclear naming, and direct file operations without abstraction.	Apply refactoring techniques to improve readability and modularity.
18	Coding Standards	Are there any violations of coding standards not covered by the checklist?	No		
19	Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist?	Yes	Direct use of System.out.println for debugging could slow down execution in production.	Replace with a configurable logging framework to manage log levels.

Checklist Analysis Table for testcases

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Limited boundary and edge case testing (e.g., null and empty cases not covered).	Add tests for boundary cases like null or empty transactionItem list.

		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	Yes		
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes		
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes		
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes		
13	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	Tests for handling null values and empty data structures (like transactionItem) are missing.	Add tests to cover edge cases, such as empty and null values for transactionItem and file inputs.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	Exception scenarios like missing files and null transactionItem list are not tested.	Add tests that check for missing files and handle exceptions gracefully.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	No mock objects are used, though they could simplify file interactions.	Use mocking for file interactions (e.g., using Mockito for

					file-based dependencies).
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	Tests depend on actual files, which may not always be ideal for unit testing.	Mock file dependencies where feasible to avoid reliance on real files.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance-related tests to verify efficiency.	Add performance tests for critical methods like retrieveTemp and endPOS.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	Yes		
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	Yes		

15.Pointofsale.java and testcases

Package Name	Default package
File Name	PointOfSale.java
Class Name	PointOfSale

Checklist Analysis Table

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in	No	Some variables like transactionItem	Rename variables to transactionItems and databaseItems for clarity.

		camelCase (e.g., calculateTotalPrice) ?		and databaseItem don't follow camelCase consistently.	
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT) ?	No	Constants like discount and tax are not in uppercase with underscores.	Rename discount to DISCOUNT and tax to TAX.
2	Code Structure	Are access modifiers (public, private, protected) used correctly according to Java standards?	Yes		
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes		
		Are packages used appropriately to organize related classes (e.g., com.pos.services)?	No	Packages are not defined in this code.	Add a package declaration, e.g., package com.pos.services;.
3	Method Design	Do methods have a single responsibility (i.e., they perform only one task)?	Yes		
		Are method parameters kept to a reasonable limit (preferably fewer than 5)?	Yes		
		Is method overloading used properly and not abused?	N/A	Not applicable as there is no method overloading in this class.	
4	Exception Handling	Are exceptions handled correctly with try-catch	No	Some file operations handle exceptions, but	Add try-catch blocks with proper exception handling

		blocks where needed?		others, like <code>removeItems</code> , could improve handling.	in methods interacting with files.
		Are specific exceptions used instead of generic <code>Exception</code> or <code>Throwable</code> ?	No	Generic <code>Exception</code> is used; specific exceptions like <code>IOException</code> could improve clarity.	Use specific exceptions like <code>IOException</code> instead of generic exceptions where possible.
		Is logging implemented within catch blocks for debugging purposes?	No	Exceptions are printed to <code>System.out</code> or ignored.	Implement proper logging using a logging framework (e.g., <code>java.util.logging</code>).
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Comments are sparse, especially in complex logic sections like <code>detectSystem</code> .	Add comments explaining the purpose of each method and complex logic.
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes		
		Are blank lines used appropriately to separate code blocks for better readability?	Yes		
		Is the code easy to read and understand?	Yes		
		Are meaningful names used for variables, classes, and methods?	No	Some variable names, like <code>unixOS</code> , are unclear in purpose.	Rename <code>unixOS</code> to <code>isUnixOS</code> or <code>isUnixBasedOS</code> .
6	Performance	Are data structures chosen based on their performance characteristics (e.g.,	Yes		

		ArrayList vs LinkedList)?			
		Are costly operations (e.g., database calls) minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	Fields like inventory are created at the class level, though they may not always be used.	Use lazy initialization for inventory if it's not always needed.
		Are there redundant calculations or excessive logging?	No	No excessive logging or redundant calculations observed.	
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	No	Unused object references are not set to null.	Set unused references to null after use to allow garbage collection.
		Are large objects or collections properly handled to avoid memory leaks (e.g., using WeakHashMap)?	No	No special handling for potential memory leaks in large collections.	Consider using WeakHashMap or clearing large collections when no longer needed.
		Are external resources like files, streams, and database connections properly closed to prevent resource leaks?	Yes	File streams are mostly closed correctly.	
		Is the try-with- resources statement used for automatic resource management (e.g.,	No	Try-with- resources is not used, making resource management more error-prone.	Use try-with-resources for resource management (e.g., in coupon).

		FileInputStream, BufferedReader)?			
		Are large files processed efficiently (e.g., using streams or buffered reading) to avoid loading the entire file into memory?	Yes		
		Is RandomAccessFile used appropriately for large files requiring specific access?	N/A	Not applicable; RandomAccessFile not required in this implementation.	
		Is error handling implemented to safely close resources in case of exceptions?	No	Some error handling is implemented, but not in a comprehensive way across all methods.	Use try-with-resources and ensure resource closure in removeItems and createTemp.
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	N/A	Not applicable in this context as there is no direct user input in the code provided.	
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	Not applicable as there is no sensitive data handling in this code.	
9	Maintainability	Are there long methods or deeply nested loops?	No	Methods are generally short with minimal nesting.	
		Is there duplicated code that could be refactored?	Yes	Duplicated code patterns for reading from files.	Refactor repeated file-reading code into a helper method.

		Are there any magic numbers or hard-coded values?	Yes	Hardcoded values like 16 for credit card length and "line.separator" appear without constants.	Replace hardcoded values with constants, e.g., CREDIT_CARD_LENGTH.
17	Housekeeping	Are there any code smells not covered by the checklist that are present in the code?	Yes	Some methods have commented-out code, which clutters readability.	Remove commented-out code or convert it to meaningful comments if necessary.
18	Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	Yes	Inconsistent access modifiers, e.g., public attributes like totalPrice should be private.	Make class attributes private and add public getter methods if necessary.
19	Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Inefficient searching in the coupon method using a linear search.	Replace linear search with a Set or another data structure for faster lookups.

Checklist Analysis Table for testcases

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes	Most public methods are tested, including startNew, enterItem, updateTotal, removeItems.	
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	Yes	Edge cases like invalid items and non-existent coupons are tested.	
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	No indication of coverage measurement in the code.	Use a coverage tool (e.g., JaCoCo) to measure code

					coverage and ensure it is maintained over 80%.
1 1	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes	Tests are organized into setup, action, and assertion sections clearly.	
		Are individual test cases independent of each other (no shared state between tests)?	Yes	Each test case restores the initial state in the tearDown method.	
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes	Test names are descriptive, e.g., testRemoveItems, testEnterItemItemNotFound.	
1 2	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes	Assertions like assertEquals, assertTrue, and assertFalse are used.	
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes	Specific assertions for different data types are used (e.g., assertTrue, assertEquals).	
1 3	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	Yes	Tests for invalid inputs like non-existing items and invalid coupons are present.	
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	Yes	testEnterItemItemNotFound and testInvalidCoupon test invalid scenarios.	
1 4	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	Mocking is not applied; code interacts directly with actual files which may slow down tests.	Use mocking frameworks (e.g., Mockito) to simulate file and database interactions.

		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	Direct file operations are used, which could be avoided with mocks.	Mock file operations and external dependencies to speed up tests and reduce resource usage.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	Performance tests for methods like <code>updateTotal</code> and <code>enterItem</code> are missing.	Add performance tests to ensure methods perform well with large data sets.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	No	Setup and teardown code is repetitive, especially for resetting files.	Move setup/teardown logic into reusable helper methods or use <code>@BeforeEach</code> and <code>@AfterEach</code> .
		Is there a setup method (<code>@BeforeEach</code>) for initializing common objects and resources used across tests?	Yes	<code>@Before</code> is used for setup, <code>@After</code> for cleanup.	

16.POR.java and testcases

Package Name	Default package
File Name	POR.java
Class Name	POR

Checklist Analysis Table

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	No	Class name POR is an abbreviation and not descriptive.	Rename to a more descriptive name, e.g., PointOfRental.
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	Follows camelCase for methods and variables.	
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	No constants are defined in uppercase.	Define constants like TEMP_FILE_PATH in uppercase format.
2	Code Structure	Are access modifiers (public, private, protected) used correctly according to Java standards?	Yes	Access modifiers are mostly used correctly.	
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes	Responsibilities are well separated.	
		Are packages used appropriately to organize related classes (e.g., com.pos.services)?	No	No package is specified.	Use a package like com.pos for better organization.
3	Method Design	Do methods have a single responsibility (i.e., they perform only one task)?	Yes	Methods generally perform one task, though endPOS could be refactored for clarity.	Refactor endPOS to separate tax calculation from inventory update.
		Are method parameters kept to a reasonable limit	Yes	Method parameters are within a reasonable limit.	

		(preferably fewer than 5)?			
		Is method overloading used properly and not abused?	Yes	No method overloading used.	
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	Yes	Exceptions are handled with try-catch, but logging is minimal.	Add detailed error messages and consider using a logging library for better traceability.
		Are specific exceptions used instead of generic Exception or Throwable?	Yes	Specific exceptions like FileNotFoundException and IOException are used.	
		Is logging implemented within catch blocks for debugging purposes?	No	Simple print statements are used instead of logging.	Replace System.out.println with a logging framework like java.util.logging for better logging.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Few comments are provided for complex logic, making the code less readable.	Add more descriptive comments, especially in methods like deleteTemplItem.
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes	Indentation is consistent.	
		Are blank lines used appropriately to separate code blocks for better readability?	Yes	Blank lines are used appropriately.	
		Is the code easy to read and understand?	No	Lack of comments and descriptive variable names in some cases makes it harder to understand.	Improve readability with comments and

					clear variable names.
		Are meaningful names used for variables, classes, and methods?	No	Variable temp and class POR are not descriptive.	Use descriptive names, e.g., temporaryFile instead of temp.
6	Performance	Are data structures chosen based on their performance characteristics (e.g., ArrayList vs LinkedList)?	Yes	ArrayList is used appropriately.	
		Are costly operations (e.g., database calls) minimized inside loops?	No	deleteTempItem method uses file handling in loops, which can be optimized.	Refactor code to reduce file operations in loops where possible.
		Is lazy initialization used to defer object creation until it is needed?	No	Objects are created at the start of methods.	Implement lazy initialization where appropriate.
		Are there redundant calculations or excessive logging?	No	No excessive logging or redundant calculations identified.	
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	No	Object references are not set to null after use.	Set object references to null in methods like deleteTempItem if no longer needed.
		Are large objects or collections properly handled to avoid memory leaks (e.g., using WeakHashMap)?	No	No handling of memory-intensive objects.	For large collections, consider alternatives like WeakHashMap.
		Are external resources like files, streams, and database connections properly closed to	Yes	Resources are closed in deleteTempItem and retrieveTemp.	

		prevent resource leaks?			
		Is the try-with-resources statement used for automatic resource management (e.g., <code>FileInputStream</code> , <code>BufferedReader</code>)?	No	Standard try-catch-finally is used instead of try-with-resources.	Use try-with-resources for better resource management.
		Are large files processed efficiently (e.g., using streams or buffered reading) to avoid loading the entire file into memory?	Yes	Buffered readers are used for reading files.	
		Is <code>RandomAccessFile</code> used appropriately for large files requiring specific access?	No	<code>RandomAccessFile</code> not used, though not necessary in this context.	Use <code>RandomAccessFile</code> only if specific access requirements arise.
		Is error handling implemented to safely close resources in case of exceptions?	Yes	Resources are closed in the catch block to prevent resource leaks.	
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	No	No user input validation implemented for phone numbers.	Validate input such as <code>phoneNum</code> to ensure it is within expected parameters.
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	No sensitive data handling is present in the code.	
9	Maintainability	Are there long methods or deeply nested loops?	No	<code>endPOS</code> method has complex logic that could be broken down.	Refactor <code>endPOS</code> into smaller,

					single-purpose methods.
		Is there duplicated code that could be refactored?	Yes	Similar file handling code in deleteTempItem and retrieveTemp.	Refactor repeated file handling code into helper methods.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded paths like "Database/newTemp.txt" and "Database/couponNumber.txt".	Use constants for paths to make the code more configurable.

Housekeeping

Category	Checklist Item	Yes/No	Issue	Fix
Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Abbreviations in class names and variables (POR, temp) make the code less readable	Use meaningful names and avoid abbreviations
Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	Yes	Inconsistent commenting practices.	Consistently document complex or unclear logic
Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Inefficient resource handling and repeated file I/O in methods	Refactor to optimize resource and file handling

Checklist Analysis Table for testcases

S#	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes	Tests cover all main methods in POR.	
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Tests do not cover null inputs or boundary values in methods like endPOS.	Add test cases for edge cases, such as null or empty inputs.

		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	No indication of coverage measurement.	Implement code coverage measurement (e.g., Jacoco) and ensure it is above 80%.
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes	Follows the AAA pattern for most tests.	
		Are individual test cases independent of each other (no shared state between tests)?	Yes	Test cases are independent due to setup and teardown methods.	
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes	Names are descriptive, indicating the functionality under test.	
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes	Assertions are used correctly to verify expected results.	
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes	Uses specific assertions like assertEquals and assertFalse.	
13	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	No edge cases such as null or empty inputs tested in endPOS or deleteTempItem.	Add tests for edge cases, e.g., empty file paths or invalid item IDs.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	No tests for invalid inputs like non-existing file paths or invalid item amounts.	Add tests for scenarios where exceptions might be thrown due to

					invalid inputs.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	No mocking used; tests directly interact with files, which could impact external systems.	Use mocking frameworks like Mockito to isolate file dependencies and avoid reliance on actual files.
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	External files are used in testing, which can lead to inconsistent results.	Mock file dependencies to ensure consistent and isolated test results.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance tests are implemented.	Add performance tests for methods with potentially heavy file I/O like retrieveTemp.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	No	Some duplication in setup and teardown, specifically with file reading and writing.	Refactor repeated file operations into helper methods to avoid duplication.
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	Yes	Setup method initializes resources and test data.	

17.POS.java and testcases

Package Name	Default package
File Name	POS.java
Class Name	POS

Checklist Analysis Table

S#	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	Constants like tempFile and tax are not in uppercase with underscores.	Rename constants to uppercase with underscores, e.g., TEMP_FILE.
2	Code Structure	Are access modifiers (public, private, protected) used correctly according to Java standards?	Yes		
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes		
		Are packages used appropriately to organize related classes (e.g., com.pos.services)?	No	No package is specified, reducing code organization and potential scalability.	Organize classes within a package structure, e.g., com.pos.
3	Method Design	Do methods have a single responsibility (i.e., they perform only one task)?	No	endPOS has multiple responsibilities, including calculating totals, updating inventory, and managing invoices.	Split endPOS into smaller, single-responsibility methods.
		Are method parameters kept to a reasonable limit (preferably fewer than 5)?	Yes		

		Is method overloading used properly and not abused?	Yes		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	Yes		
		Are specific exceptions used instead of generic Exception or Throwable?	No	Generic IOException and FileNotFoundException used without specifying the exact cause in some cases.	Implement specific exceptions where possible to handle distinct cases, or add descriptive log messages.
		Is logging implemented within catch blocks for debugging purposes?	No	Logging is not implemented; instead, some System.out messages are used.	Replace System.out with logging (e.g., Logger) to capture exceptions.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Some areas, like endPOS, could benefit from more detailed comments explaining the complex logic and flow.	Add comments explaining key logic in complex methods, especially endPOS.
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes		
		Are blank lines used appropriately to separate code blocks for better readability?	Yes		
		Is the code easy to read and understand?	No	Nested conditionals in endPOS affect readability.	Refactor complex nested structures into separate methods for clarity.

		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on their performance characteristics (e.g., ArrayList vs LinkedList)?	Yes		
		Are costly operations (e.g., database calls) minimized inside loops?	Yes		
		Is lazy initialization used to defer object creation until it is needed?	No	transactionItem and databaseItem are initialized even when not necessarily required.	Use lazy initialization for variables where possible to optimize memory usage.
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	No	Object references for databaseItem and transactionItem could be set to null after use in endPOS.	Set object references to null after use to facilitate garbage collection.
		Are large objects or collections properly handled to avoid memory leaks (e.g., using WeakHashMap)?	N/A		
		Are external resources like files, streams, and database connections properly closed to prevent resource leaks?	Yes		
		Is the try-with-resources statement	No	Try-with-resources not used, leading to more	Use try-with-resources for

		used for automatic resource management (e.g., FileInputStream, BufferedReader)?		verbose and potentially error-prone resource management.	automatic resource closing where possible.
		Are large files processed efficiently (e.g., using streams or buffered reading) to avoid loading the entire file into memory?	Yes		
		Is RandomAccessFile used appropriately for large files requiring specific access?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	Yes		
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	N/A		
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A		
9	Maintainability	Are there long methods or deeply nested loops?	Yes	endPOS contains deeply nested loops and conditionals, making it hard to maintain.	Refactor endPOS into smaller methods to improve readability and maintainability.
		Is there duplicated code that could be refactored?	Yes	Similar code patterns in file handling across methods (deleteTempItem and retrieveTemp).	Refactor repetitive code into helper methods for better maintainability.

		Are there any magic numbers or hard-coded values?	Yes	Hardcoded strings like file paths (e.g., Database/newTemp.txt).	Replace hardcoded values with constants or configurable properties.
17	Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Use of System.out for error handling and deeply nested conditionals in endPOS are potential code smells.	Replace System.out with logging and refactor nested conditionals.
18	Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	Yes	Lack of package declaration, inconsistent exception handling (generic exception types used).	Add package declarations and replace generic exceptions with specific types.
19	Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	Yes	Inefficiencies in memory usage (e.g., holding unnecessary references) and redundant instantiation in loops.	Optimize memory usage and instantiation by setting unused references to null and reusing objects where possible.

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Limited edge case coverage; tests primarily check specific values without boundary conditions.	Add tests for boundary cases, e.g., empty transactionItem or invalid file paths.

		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	No evidence of code coverage measurement in the test suite.	Use a code coverage tool (e.g., JaCoCo) to ensure adequate test coverage.
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes		
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes		
1 2	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes		
1 3	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., max/min values) tested?	No	Tests for null or empty transactionItem or invalid file scenarios are missing.	Add tests for null values, empty lists, and boundary conditions.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	Exceptional scenarios like FileNotFoundException or null checks in input handling are not tested.	Add tests for exception handling with invalid inputs.
1 4	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	Dependencies are not mocked; real file operations are used, which may impact test reliability.	Use mocking (e.g., Mockito) to avoid actual

					file operations during tests.
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	File dependencies are not mocked, making the tests dependent on actual files, reducing isolation.	Use mock files or dependency injection to isolate tests from actual files.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance tests implemented for critical methods.	Consider adding performance tests for critical methods if needed.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	No	The readFileContent method is duplicated; the same logic is repeated in different tests.	Refactor common code like readFileContent into a helper method or utility class.
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	Yes		

18. POSSystem.java and testcases

Package Name	Default package
File Name	POSSystem.java
Class Name	POSSystem

Checklist Analysis Table

S #	Category	Checklist Item	Yes/No	Issue	Fix
-----	----------	----------------	--------	-------	-----

1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	No	Some variable names (e.g., cal, textReader) are not descriptive.	Use more descriptive camelCase names (e.g., calendar, bufferedTextReader).
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	No	Constant String paths like employeeDatabase are not uppercase.	Use uppercase with underscores for constants (e.g., EMPLOYEE_DATABASE).
2	Code Structure	Are access modifiers used correctly according to Java standards?	No	Some methods, like readFile, are package-private without a clear reason.	Define access modifiers explicitly, ideally as private or protected.
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes		
		Are packages used appropriately to organize related classes?	No	Classes do not appear to be organized into packages.	Organize classes under appropriate packages (e.g., com.pos.system).
3	Method Design	Do methods have a single responsibility (i.e., they perform only one task)?	No	logIn and logOut methods handle multiple tasks such as authentication and file logging.	Split into methods for authenticateUser and logToFile.
		Are method parameters kept to a reasonable limit (preferably fewer than 5)?	Yes		
		Is method overloading used properly and not abused?	N/A		
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	Yes		

		Are specific exceptions used instead of generic Exception or Throwable?	Yes		
		Is logging implemented within catch blocks for debugging purposes?	No	Missing logging for some exceptions.	Add logging statements in catch blocks to provide more context on errors.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	Few or no comments explaining complex logic (e.g., OS check, method responsibilities)	Add comments to explain complex logic and purpose of the methods.
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes		
		Are blank lines used appropriately to separate code blocks for better readability?	Yes		
		Is the code easy to read and understand?	No	Some methods lack descriptive comments and variable names are occasionally unclear.	Use descriptive names and add comments where necessary.
		Are meaningful names used for variables, classes, and methods?	No	Some variables like lineSort and cal are not fully descriptive.	Use descriptive names that convey the purpose (e.g., sortedLine, calendarInstance).
6	Performance	Are data structures chosen based on their performance characteristics?	Yes		
		Are costly operations (e.g., database calls) minimized inside loops?	No	Database operations are handled once, but repeated	Optimize file access to reduce multiple read operations within methods.

				file reads could be avoided.	
		Is lazy initialization used to defer object creation until it is needed?	Yes		
		Are there redundant calculations or excessive logging?	No		
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	No	Objects like textReader and fileR are not explicitly set to null after use.	Set references to null after closing them to aid garbage collection.
		Are large objects or collections properly handled to avoid memory leaks?	Yes		
		Are external resources like files, streams, and database connections properly closed?	Yes		
		Is the try-with-resources statement used for automatic resource management?	No	Uses try-catch without try-with-resources for file operations.	Use try-with-resources for file readers/writers to ensure they are closed automatically.
		Are large files processed efficiently (e.g., using streams or buffered reading)?	Yes		
		Is RandomAccessFile used appropriately for large files requiring specific access?	N/A		
		Is error handling implemented to safely close resources in case of exceptions?	No	Files are closed in finally, but try-with-resources would be safer.	Use try-with-resources instead of manual closing to improve safety.
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	No	No input validation observed for username and	Add input validation to ensure only valid characters are used in

				password in logIn.	usernames and passwords.
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	No	Passwords are stored in plaintext and are not hashed or encrypted.	Use hashing (e.g., BCrypt) to securely store passwords.
9	Maintainability	Are there long methods or deeply nested loops?	No	Some methods, such as readFile, handle multiple tasks and are quite long.	Refactor longer methods like readFile into smaller methods for readability.
		Is there duplicated code that could be refactored?	Yes	Some code for OS detection and file access is duplicated in multiple methods.	Refactor duplicated code into separate helper methods.
		Are there any magic numbers or hard-coded values?	Yes	Hard-coded file paths and other values are not parameterized or configurable.	Use constants or configuration files for managing paths and other constants.
17	Code Smells	Are there any code smells not covered by the checklist that are present in the code?	Yes	Hard-coded paths and OS detection logic contribute to code smells.	Refactor paths to constants or configurations and reduce OS checks if possible.
18	Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	Yes	Non-standard naming of certain variables and a mix of responsibilities within methods.	Ensure methods adhere to single responsibility and use Java naming conventions.
19	Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	File operations are not optimized for efficiency, and repetitive access might cause	Consider batching file operations or caching frequently accessed data if needed.

				performance hits.	
--	--	--	--	-------------------	--

Checklist Analysis Table for testcases

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	No	Not all functionalities are covered (e.g., logOutToFile, logInToFile).	Add tests for each method in POSSystem to improve coverage.
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Some edge cases (e.g., null inputs for logIn, empty files) are missing.	Add tests for boundary cases, such as null or invalid inputs.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	Coverage not specified, and multiple methods lack testing.	Ensure coverage meets or exceeds 80% by testing all possible scenarios.
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes		
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes		

1 2	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes		
1 3	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	Not all edge cases are covered, such as invalid paths or unexpected file contents.	Test for edge cases, including unexpected or corrupt file contents and invalid parameter values.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	Invalid input handling (e.g., non-existent files) is not tested.	Test for invalid file paths and exceptions, such as FileNotFoundException.
1 4	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	No mocking used, causing dependencies on real files and I/O.	Use mocking frameworks (e.g., Mockito) to mock file interactions and database dependencies.
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	No	Tests depend on the actual file system, which can make tests unreliable.	Replace file dependencies with mocks or use an in-memory database for testing.
1 5	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance or stress tests included.	Add performance tests for critical paths, such as continueFromTemp and login.
1 6	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	No	Some duplicated setup logic (e.g., copying	Modularize test setup and teardown code by refactoring into helper methods.

				files) exists across tests.	
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	Yes		

19.ReturnItem.java and testcases

Package Name	Default package
File Name	ReturnItem.java
Class Name	ReturnItem

Checklist Analysis Table

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes		
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes		
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	N/A	No constants are present in this class.	No action needed.
2	Code Structure	Are access modifiers (public, private, protected) used correctly according to Java standards?	Yes		
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes		
		Are packages used appropriately to organize related classes (e.g., com.pos.services)?	No	No package statement is included, leading to potentially unorganized code in larger projects.	Add a package declaration such as package com.pos.items; .

3	Method Design	Do methods have a single responsibility (i.e., they perform only one task)?	Yes		
		Are method parameters kept to a reasonable limit (preferably fewer than 5)?	Yes		
		Is method overloading used properly and not abused?	N/A	No method overloading is present.	No action needed.
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	N/A	No exception handling is necessary in this class.	No action needed.
		Are specific exceptions used instead of generic Exception or Throwable?	N/A	No exception handling is present.	No action needed.
		Is logging implemented within catch blocks for debugging purposes?	N/A	No logging is needed in this class.	No action needed.
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	No comments are present; could benefit from basic comments explaining the purpose of the class and methods.	Add comments for class and method descriptions, e.g., <code>// Represents an item returned with days since return.</code>
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes		
		Are blank lines used appropriately to separate code blocks for better readability?	Yes		
		Is the code easy to read and understand?	Yes		

		Are meaningful names used for variables, classes, and methods?	Yes		
6	Performance	Are data structures chosen based on their performance characteristics (e.g., ArrayList vs LinkedList)?	N/A	Not applicable as there are no complex data structures used.	No action needed.
		Are costly operations (e.g., database calls) minimized inside loops?	N/A	Not applicable as there are no loops or costly operations in this class.	No action needed.
		Is lazy initialization used to defer object creation until it is needed?	N/A	Not applicable in this simple class.	No action needed.
		Are there redundant calculations or excessive logging?	N/A	No redundant calculations or logging is present.	No action needed.
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	N/A	Not applicable as there is no complex memory management required.	No action needed.
		Are large objects or collections properly handled to avoid memory leaks (e.g., using WeakHashMap)?	N/A	No collections are used.	No action needed.
		Are external resources like files, streams, and database connections properly closed to prevent resource leaks?	N/A	No external resources are used.	No action needed.

		Is the try-with-resources statement used for automatic resource management (e.g., FileInputStream, BufferedReader)?	N/A	Not applicable as no resources are opened.	No action needed.
		Are large files processed efficiently (e.g., using streams or buffered reading) to avoid loading the entire file into memory?	N/A	Not applicable as no file processing occurs here.	No action needed.
		Is RandomAccessFile used appropriately for large files requiring specific access?	N/A	Not applicable here.	No action needed.
		Is error handling implemented to safely close resources in case of exceptions?	N/A	No resources to close.	No action needed.
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	N/A	Not applicable as this class does not handle user input.	No action needed.
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	No sensitive data is handled by this class.	No action needed.
9	Maintainability	Are there long methods or deeply nested loops?	No		
		Is there duplicated code that could be refactored?	No		
		Are there any magic numbers or hard-coded values?	No		
17	Housekeeping - Code Smells	Are there any code smells not covered by the checklist that are present in the code?	No		
18	Housekeeping - Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No		

19	Housekeeping - Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No		
----	---	---	----	--	--

S #	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Tests do not cover edge cases, such as null inputs or invalid values.	Add tests for edge cases like null input or values like 0 or negative days.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	No information about code coverage measurement is provided.	Use a code coverage tool to ensure the coverage level, aiming for at least 80%.
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes		
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes		
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	N/A	Only basic assertions like assertEquals are required	No action needed.

				here, which is appropriate.	
13	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	No tests for minimum values, such as 0 or negative days.	Add test cases to cover boundary and edge cases.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	Tests do not include handling for invalid input or exceptions.	Add tests for invalid inputs and ensure handling for potential exceptions.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	N/A	No dependencies to mock in this test class.	No action needed.
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	N/A		No action needed.
15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	N/A	Not applicable for this basic class.	No action needed.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	Yes		
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	No	setUp() and tearDown() methods are present but unused.	Remove unused setup/tearDown methods or use them to initialize common objects.

20.Register.java and testcases

Package Name	Default package
File Name	Register.java
Class / Interface Name	Register

S #	Category	Checklist Item	Yes/No	Issue	Fix
1	Naming Conventions	Are class names written in PascalCase (e.g., OrderService)?	Yes	-	-
		Are variable and method names written in camelCase (e.g., calculateTotalPrice)?	Yes	-	-
		Are constants written in uppercase with underscores (e.g., MAX_DISCOUNT)?	N/A	No constants are defined	Define constants in uppercase with underscores if added in future
2	Code Structure	Are access modifiers (public, private, protected) used correctly according to Java standards?	Yes	-	-
		Are classes and interfaces clearly separated (e.g., no mixed responsibilities)?	Yes	-	-
		Are packages used appropriately to organize related classes (e.g., com.pos.services)?	No	Package is not specified	Add package declaration to organize the code

3	Method Design	Do methods have a single responsibility (i.e., they perform only one task)?	Yes	-	-
		Are method parameters kept to a reasonable limit (preferably fewer than 5)?	Yes	-	-
		Is method overloading used properly and not abused?	N/A	Only a single main method is defined	-
4	Exception Handling	Are exceptions handled correctly with try-catch blocks where needed?	No	No error handling for setting default close operation	Add try-catch for setDefaultCloseOperation to handle potential exceptions
		Are specific exceptions used instead of generic Exception or Throwable?	N/A	No exceptions are handled	-
		Is logging implemented within catch blocks for debugging purposes?	N/A	No catch blocks are present	-
5	Code Readability	Are meaningful and descriptive comments added for complex logic?	No	No comments provided	Add comments to explain complex or critical parts
		Is there consistent indentation (4 spaces or a tab size of 4)?	Yes	-	-
		Are blank lines used appropriately to separate code blocks for better readability?	Yes	-	-

		Is the code easy to read and understand?	Yes	-	-
		Are meaningful names used for variables, classes, and methods?	Yes	-	-
6	Performance	Are data structures chosen based on their performance characteristics (e.g., ArrayList vs LinkedList)?	N/A	No data structures used in the class	-
		Are costly operations (e.g., database calls) minimized inside loops?	N/A	No loops or costly operations present	-
		Is lazy initialization used to defer object creation until it is needed?	N/A	No object creation issues	-
		Are there redundant calculations or excessive logging?	N/A	No calculations or logging present	-
7	Memory Management	Are unnecessary object references set to null to aid garbage collection?	N/A	No object references handled	-
		Are large objects or collections properly handled to avoid memory leaks (e.g., using WeakHashMap)?	N/A	No large objects or collections used	-
		Are external resources like files, streams, and database connections properly closed to	N/A	No external resources used	-

		prevent resource leaks?			
		Is the try-with-resources statement used for automatic resource management (e.g., FileInputStream, BufferedReader)?	N/A	No resource handling is required	-
		Are large files processed efficiently (e.g., using streams or buffered reading) to avoid loading the entire file into memory?	N/A	No file handling in the class	-
		Is RandomAccessFile used appropriately for large files requiring specific access?	N/A	No file access needed	-
		Is error handling implemented to safely close resources in case of exceptions?	N/A	No resources to close	-
8	Security	Is user input validated to prevent security issues (e.g., SQL injection, XSS)?	N/A	No user input is handled in this class	-
		Are sensitive data (e.g., passwords) encrypted or hashed before storage?	N/A	No sensitive data handling	-
9	Maintainability	Are there long methods or deeply nested loops?	No	-	-

		Is there duplicated code that could be refactored?	No	No duplicated code	-
		Are there any magic numbers or hard-coded values?	Yes	Hardcoded DISPOSE_ON_CLOSE	Define as constant variable
17	House Keeping	Are there any code smells not covered by the checklist that are present in the code?	No	-	-
18	Coding Standards	Are there any violations of coding standards not covered by the checklist that are present in the code?	No	-	-
19	Performance Inefficiencies	Are there any performance inefficiencies not covered by the checklist that are present in the code?	No	-	-

Checklist for test cases:

S#	Category	Checklist Item	Yes/No	Issue	Fix
10	Test Coverage	Are unit tests provided for all public methods and critical functionalities?	Yes		
		Do the unit tests cover all branches and edge cases (e.g., boundary values, null inputs)?	No	Tests for boundary cases, null input cases are missing.	Add tests to cover cases where loginInterface might be null or in different UI states.
		Is code coverage measured and maintained at an acceptable level (e.g., 80% or higher)?	No	No code coverage	Use tools like JaCoCo or Cobertura to

				measurement mentioned.	measure coverage.
11	Test Design	Are tests written to follow the AAA pattern (Arrange, Act, Assert) for clarity and consistency?	Yes		
		Are individual test cases independent of each other (no shared state between tests)?	Yes		
		Are meaningful and descriptive names used for test methods (e.g., testCalculateTotal_WithDiscount)?	Yes		
12	Assertions	Are assertions used to verify expected results (assertEquals, assertTrue, assertThrows)?	Yes		
		Are specific assertions used instead of general ones (e.g., using assertEquals for arrays)?	Yes		
13	Boundary and Edge Cases	Are edge cases (e.g., empty input, null values) and boundary conditions (e.g., maximum/minimum values) tested?	No	Edge cases like null values for loginInterface are not covered.	Add tests for scenarios where loginInterface could be null or interact in edge cases.
		Are invalid inputs and exceptional scenarios covered by tests (e.g., handling NullPointerException)?	No	No checks for null cases or exceptions in the UI component tests.	Include tests for invalid states or configurations of loginInterface.
14	Mocking and Stubbing	Are mocks or stubs used to isolate the unit under test (e.g., using Mockito for dependencies)?	No	Mocking is not applied where it could enhance testing, like UI interactions.	Use Mockito to mock loginInterface or similar components if dependencies exist.
		Are dependencies (e.g., database connections, web services) mocked to avoid external calls?	Yes	Not applicable in this UI test context.	

15	Performance Testing	Are there tests to check performance for critical methods (e.g., response time, execution speed)?	No	No performance testing, but minimal for a simple UI component.	Add basic performance tests if necessary, though not critical for this UI functionality.
16	Test Maintainability	Are test methods well-organized and modular, avoiding duplication of code?	Yes		
		Is there a setup method (@BeforeEach) for initializing common objects and resources used across tests?	Yes		

B. Refactoring

1. POH class

Identified Area for Refactoring:

The most impactful refactoring will address **Code Structure, Method Design, Exception Handling, and Code Smells**. Specifically, focusing on:

1. **Method deleteTempItem:**

- It mixes multiple responsibilities such as reading a file, writing a file, and updating items.
- Exception handling uses `System.out.println`, which is not suitable for production.

2. **Method endPOS:**

- It is lengthy and includes nested loops, which hinder readability.
- Hardcoded values like 0.1 (late fee multiplier) and file paths need constants.

3. **Method retrieveTemp:**

- It combines inventory access, file reading, and business logic, violating the single-responsibility principle.

Refactoring Plan:

1. **Extract Reusable Helper Methods**

- Separate file operations (read/write) into reusable helper methods.
- Abstract repeated operations like file path handling.

2. Adopt Logging Framework

- Replace System.out.println with a logging framework like java.util.logging.

3. Constants and Naming Conventions

- Define constants for hardcoded values like paths and multipliers.
- Rename variables and methods for clarity.

4. Improve Exception Handling

- Use try-with-resources for resource management.
- Add meaningful messages to logs.

Impact Analysis

1. Improved Readability:

- Clear variable and method names.
- Modular code for reusability.

2. Enhanced Performance:

- Eliminated redundant operations like scattered System.out.println.
- Abstracted file operations to helpers.

3. Better Maintainability:

- Reduced method complexity.
- Constants for configurable values.

4. Robust Exception Handling:

- Replaced manual resource management with try-with-resources.
- Logging framework adds flexibility for debugging and production.

Before code

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class POH extends PointOfSale
{
    List <ReturnItem> returnList = new ArrayList<ReturnItem>();
    long phone;
```

```

public POH(){
    this.phone=0;
}

public POH(long phone)
{
    this.phone = phone;
}


public void deleteTempItem(int id){
    try{
        String temp = "Database/newTemp.txt";
        if(System.getProperty("os.name").startsWith("W")||System.getProperty("os.
name").startsWith("w")){
            // temp = "..\\Database\\newTemp.txt";
        }
        File tempF = new File(temp);
        FileReader fileR = new FileReader(tempFile);
        BufferedReader reader = new BufferedReader(fileR);
        BufferedWriter writer = new BufferedWriter(new FileWriter(tempF));
        String type= reader.readLine();
        String phone;
        phone=reader.readLine();
        writer.write(type);
        writer.write(System.getProperty("line.separator"));
        writer.write(phone);
        writer.write(System.getProperty("line.separator"));
        for (int i =0; i<transactionItem.size();i++){
            if (transactionItem.get(i).getItemID()!=id){
                writer.write(transactionItem.get(i).getItemID() +" "+
transactionItem.get(i).getAmount());
                writer.write(System.getProperty( "line.separator" ));
            }
        }
        fileR.close();
        writer.close();
        reader.close();
        File file = new File(tempFile);
        file.delete();
        tempF.renameTo(new File(tempFile));
    }
}

```

```

    }
    catch(FileNotFoundException ex) {
        System.out.println(
            "Unable to open file 'temp'");
    }
    catch(IOException ex) {
        System.out.println(
            "Error reading file 'temp'");
    }
}

public double endPOS(String textFile){
    //detectSystem();
    if(returnSale==true){
        try{
            String t = "Database/returnSale.txt";

            FileWriter fw2 = new FileWriter(t,true);
            BufferedWriter bw2 = new BufferedWriter(fw2);
            bw2.write(System.getProperty( "line.separator" ));
            for(int i=0;i<transactionItem.size();i++){
                String log=Integer.toString(transactionItem.get(i).getItemID())+"
"+transactionItem.get(i).getItemName()+" "+
                    Integer.toString(transactionItem.get(i).getAmount())+" "+
                    Double.toString(transactionItem.get(i).getPrice()*transac
tionItem.get(i).getAmount());
                bw2.write(log);
                bw2.write(System.getProperty( "line.separator" ));
            }
            bw2.newLine();
            bw2.close();

        } catch (FileNotFoundException e) {
            System.out.println("Unable to open file Log File for logout");
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
    if (transactionItem.size() > 0 && !textFile.equals(""))
    {
        Management management = new Management();
        returnList = management.getLatestReturnDate(phone);
        double itemPrice = 0;
        // totalPrice = 0;
    }
}

```

```

        for (int transactionCounter = 0; transactionCounter <
transactionItem.size(); transactionCounter++)
            for (int returnCounter = 0; returnCounter < returnList.size();
returnCounter++)
            {
                if (transactionItem.get(transactionCounter).getItemID() ==
returnList.get(returnCounter).getItemID())
                {
                    //Applies a value to be payed depending on the amount of days it
is late. If it is not late, no value is applied
                    itemPrice =
transactionItem.get(transactionCounter).getAmount()*transactionItem.get(transacti
onCounter).getPrice()* 0.1 * returnList.get(returnCounter).getDays();
                    totalPrice += itemPrice;
                    System.out.println("Item Name: " +
transactionItem.get(transactionCounter).getItemName() + "    Days Late: "
+ returnList.get(returnCounter).getDays() +
"    To be paid: " + itemPrice);
                    System.out.println("Total: " + totalPrice);
                }
            }

        inventory.updateInventory(textFile, transactionItem, databaseItem,false);

        management.updateRentalStatus(phone,returnList);

    }

    databaseItem.clear();
    transactionItem.clear();
    return totalPrice;
}

public void retrieveTemp(String textFile){
    try{
        FileReader fileR = new FileReader(tempFile);
        BufferedReader textReader = new BufferedReader(fileR);
        String line=null;
        String[] lineSort;

```

```

        line=textReader.readLine();
        inventory.accessInventory(textFile, databaseItem);
        line=textReader.readLine();
        System.out.println("Phone number:");
        System.out.println(line);

        while ((line = textReader.readLine()) != null)
        {
            lineSort = line.split(" ");
            int itemNo = Integer.parseInt(lineSort[0]);
            int itemAmount = Integer.parseInt(lineSort[1]);
            enterItem(itemNo,itemAmount);
        }

        updateTotal();
        textReader.close();
    }
    catch(FileNotFoundException ex) {
        System.out.println(
            "Unable to open file 'temp'");
    }
    catch(IOException ex) {
        System.out.println(
            "Error reading file 'temp'");
    }
}
}

```

After code

```

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.logging.Level;
import java.util.logging.Logger;

public class POH extends PointOfSale {

    private static final Logger LOGGER = Logger.getLogger(POH.class.getName());
    private static final String TEMP_FILE_PATH = "Database/newTemp.txt";
    private static final String RETURN_SALE_FILE_PATH =
        "Database/returnSale.txt";
    private static final double LATE_FEE_MULTIPLIER = 0.1;

```

```

List<ReturnItem> returnList = new ArrayList<>();
long phone;

public POH() {
    this.phone = 0;
}

public POH(long phone) {
    this.phone = phone;
}

// Delete a temporary item based on item ID
public void deleteTempItem(int id) {
    File tempFile = new File(TEMP_FILE_PATH);
    File newTempFile = new File("Database/tempNew.txt");

    try (BufferedReader reader = new BufferedReader(new
FileReader(tempFile));
        BufferedWriter writer = new BufferedWriter(new
FileWriter(newTempFile))) {

        // Copy type and phone to the new file
        writer.write(reader.readLine() + System.lineSeparator());
        writer.write(reader.readLine() + System.lineSeparator());

        // Copy other items, excluding the one to delete
        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(" ");
            int itemId = Integer.parseInt(parts[0]);
            if (itemId != id) {
                writer.write(line + System.lineSeparator());
            }
        }

        // Replace old file with the updated one
        if (!tempFile.delete() || !newTempFile.renameTo(tempFile)) {
            LOGGER.warning("Failed to replace temp file.");
        }

    } catch (IOException e) {
        LOGGER.log(Level.SEVERE, "Error handling temporary file operations",
e);
    }
}

```

```

    }

    // End Point-of-Sale process
    public double endPOS(String textFile) {
        double totalPrice = 0;

        if (returnSale) {
            appendReturnSaleLog();
        }

        if (!transactionItem.isEmpty() && !textFile.isEmpty()) {
            Management management = new Management();
            returnList = management.getLatestReturnDate(phone);

            for (ReturnItem returnItem : returnList) {
                for (Item transaction : transactionItem) {
                    if (transaction.getItemID() == returnItem.getItemID()) {
                        double itemPrice = calculateLateFee(transaction,
returnItem);

                        totalPrice += itemPrice;
                        LOGGER.info(String.format("Item: %s, Days Late: %d, Fee:
%.2f",
                                                transaction.getItemName(), returnItem.getDays(),
itemPrice));
                    }
                }
            }

            inventory.updateInventory(textFile, transactionItem, databaseItem,
false);
            management.updateRentalStatus(phone, returnList);
        }

        transactionItem.clear();
        databaseItem.clear();
        return totalPrice;
    }

    private void appendReturnSaleLog() {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(RETURN_SALE_FILE_PATH, true))) {
            writer.newLine();
            for (Item transaction : transactionItem) {
                writer.write(String.format("%d %s %d %.2f%n",
transaction.getItemID(),

```

```

        transaction.getItemName(),
        transaction.getAmount(),
        transaction.getPrice() * transaction.getAmount()));
    }
} catch (IOException e) {
    LOGGER.log(Level.SEVERE, "Error writing return sale log", e);
}
}

private double calculateLateFee(Item transaction, ReturnItem returnItem) {
    return transaction.getAmount() * transaction.getPrice() *
LATE_FEE_MULTIPLIER * returnItem.getDays();
}

// Retrieve temporary data
public void retrieveTemp(String textFile) {
    try (BufferedReader reader = new BufferedReader(new
FileReader(TEMP_FILE_PATH))) {
        inventory.accessInventory(textFile, databaseItem);

        LOGGER.info("Phone number: " + reader.readLine());

        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(" ");
            int itemId = Integer.parseInt(parts[0]);
            int itemAmount = Integer.parseInt(parts[1]);
            enterItem(itemId, itemAmount);
        }

        updateTotal();

    } catch (IOException e) {
        LOGGER.log(Level.SEVERE, "Error reading temporary file", e);
    }
}
}

```

2. POSSystems class

Identified Area for Refactoring

The readFile method is a prime candidate for refactoring:

- It combines file reading, parsing, and employee list population into one long method.

- Naming conventions are not descriptive (lineSort, cal).
- Access modifiers and exception handling are suboptimal.
- File operations can benefit from **try-with-resources** to improve resource management and code readability.

Proposed Refactoring Steps

1. **Break Down Responsibilities:**

- Extract file reading logic into a helper method (readEmployeeDatabase).
- Use a dedicated method to parse lines into Employee objects (parseEmployee).

2. **Improve Naming:**

- Rename variables to better reflect their purpose (lineSort → employeeDetails, textReader → bufferedReader).

3. **Enhance Exception Handling:**

- Add proper logging for exceptions using meaningful messages.
- Use try-with-resources for automatic file closure.

4. **Make Constants Uppercase:**

- Convert file paths to uppercase constants.

5. **Organize Classes into Packages:**

- Suggest placing classes into a structured package hierarchy (com.pos.system).

Changes Made

1. **File Reading Refactored:**

- Separated file reading into readEmployeeDatabase.
- Added parseEmployee for parsing individual lines.

2. **Improved Naming:**

- Updated variable names for better clarity (lineSort → employeeDetails).

3. **Used try-with-resources:**

- Ensures file resources are automatically closed.

4. **Constants Uppercased:**

- Updated file path constants to uppercase.

5. **Enhanced Logging:**

- Added meaningful error messages for exceptions.

6. **Single Responsibility Methods:**

- Split logging into a reusable logActionToFile method.
-

Impact Analysis

1. **Code Readability:**

- Improved clarity with descriptive variable names and smaller, focused methods.

2. **Error Handling:**

- Reduced likelihood of resource leaks with try-with-resources.
- Added meaningful error messages for better debugging.

3. **Maintainability:**

- Separation of concerns simplifies future enhancements or debugging.

4. **Performance:**

- Slight improvement in file handling due to optimized resource management.

5. **Security:**

- No direct improvement in password storage; hashing should be implemented in future iterations.

Before code

```
import java.text.DateFormat;
import java.text.SimpleDateFormat;
import java.util.*;
import java.io.*;
import java.io.FileNotFoundException;

public class POSSystem{
    public boolean unixOS = true;
    public static String employeeDatabase = "Database/employeeDatabase.txt";
    public static String rentalDatabaseFile = "Database/rentalDatabase.txt";
    public static String itemDatabaseFile = "Database/itemDatabase.txt";
    public List<Employee> employees = new ArrayList<Employee>();
    DateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss.SSS");
    Calendar cal=null;
    private int index=-1;
    String username="";
    String password="";
    String name="";

    void readFile(){
```

```

        if
(System.getProperty("os.name").startsWith("W")||System.getProperty("os.name").sta
rtsWith("w")){
            //unixOS = false; //commented out to support netbeans
            //employeeDatabase = "..\\Database\\employeeDatabase.txt";
            //rentalDatabaseFile = "..\\Database\\rentalDatabase.txt";
            //itemDatabaseFile = "..\\Database\\itemDatabase.txt";
        }

String line = null;
String[] lineSort;

//Checks database file for the item
try {
    FileReader fileR = new FileReader(employeeDatabase);
    BufferedReader textReader = new BufferedReader(fileR);
    //reads the entire database
    while ((line = textReader.readLine()) != null)
    {
        lineSort = line.split(" "); //separates words
        String name=lineSort[2]+" "+lineSort[3];
        employees.add(new Employee(lineSort[0],name,lineSort[1],lineSort[4]));
    }
    textReader.close();

}

//catches exceptions
catch(FileNotFoundException ex) {
    System.out.println(
        "Unable to open file '" +
        employeeDatabase + "'");
}
catch(IOException ex) {
    System.out.println(
        "Error reading file '"
        + employeeDatabase + "'");
}
}

private void logInToFile(String username,String name,String position,Calendar
cal){
    try{
        String temp = "Database/employeeLogfile.txt";

```

```

        //if(System.getProperty("os.name").startsWith("W")||System.getProperty("os.
name").startsWith("w")){
            //temp = "..\\Database\\employeeLogfile.txt"; //commented out to support
netbeans
        //}
        FileWriter fw = new FileWriter(temp,true);
        BufferedWriter bw = new BufferedWriter(fw);
        String log=name+" (" +username+" "+position+" ) logs into POS System. Time:
"+dateFormat.format(cal.getTime());
        bw.write(log);
        bw.write(System.getProperty( "line.separator" ));
        bw.close();
        if(position.equals("Admin")){

        }

    } catch (FileNotFoundException e) {
        System.out.println("Unable to open file Log File for logIn");
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}

public boolean checkTemp()
{
    String temp = "Database/temp.txt";
    File f=new File(temp);
    if(f.exists() && !f.isDirectory())
        return true;
    return false;
}

public String continueFromTemp(long phone)
{
    String temp = "Database/temp.txt";
    File f=new File(temp);

    try{
        FileReader fileR = new FileReader(temp);
        BufferedReader textReader = new BufferedReader(fileR);
        if(f.length()==0){
            System.out.println("The log file is not valid");
            f.delete();
        }
    }
}

```

```

        else{
            String type= textReader.readLine();
            if(type.equals("Sale")){
                //System.out.println("Sale");
                POS sale=new POS();
                sale.retrieveTemp(itemDatabaseFile);
                textReader.close();
                return "Sale";
            }
            else if(type.equals("Rental")){
                //System.out.println("Rental");
                POR rental=new POR(phone);
                rental.retrieveTemp(rentalDatabaseFile);
                textReader.close();
                return "Rental";
            }

            else if(type.equals("Return")){
                //System.out.println("Return");
                POH returns = new POH(phone);
                returns.retrieveTemp(rentalDatabaseFile);
                textReader.close();
                return "Return";
            }

        }
        textReader.close();
    }
    catch(FileNotFoundException ex) {
        //System.out.println(
            //            "Unable to open file 'temp'");
    }
    catch(IOException ex) {
        //System.out.println(
            //            "Error reading file 'temp'");
        //ableToOpen = false;
    }
    return "";
}

private void logOutToFile(String username,String name,String position,Calendar
cal){
    try{
        String temp = "Database/employeeLogfile.txt";
    }
}

```

```

        if(System.getProperty("os.name").startsWith("W")||System.getProperty("os.name").startsWith("w")){
            //temp = "..\\Database\\employeeLogfile.txt";
        }
        FileWriter fw = new FileWriter(temp,true);
        BufferedWriter bw = new BufferedWriter(fw);
        String log=name+" (" +username+" "+position+" ) logs out of POS System. Time: "+dateFormat.format(cal.getTime());
        bw.write(log);
        bw.write(System.getProperty( "line.separator" ));
        bw.close();

    } catch (FileNotFoundException e) {
        System.out.println("Unable to open file Log File for logout");
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}

public void logOut(String pos){
    cal = Calendar.getInstance();
    logOutToFile(username,name,pos,cal);
}

public int logIn(String userAuth, String passAuth){
    readFile();
    username=userAuth;
    boolean find=false;
    for(int i=0;i<employees.size();i++){
        if(username.equals((employees.get(i)).getUsername())){
            find=true;
            index=i;
            break;
        }
    }
    if (find == true)
    {
        password=passAuth;
        if(!password.equals((employees.get(index)).getPassword())){
            return 0; //didn't find employee password
        }
        else{
            //employee logIn file update

```

```

        cal = Calendar.getInstance();
        name=(employees.get(index)).getName();
        logInToFile((employees.get(index)).getUsername(),name,(employees.get(index)).getPosition(),cal);

        if(((employees.get(index)).getPosition()).equals("Cashier")){
            return 1; //returns cashier status
        }
        else if(((employees.get(index)).getPosition()).equals("Admin")){
            return 2; //returns admin status
        }
    }
}
return 0;//didnt find employee username
}
}

```

After code

```

import java.io.*;
import java.text.DateFormat;
import java.text.SimpleDateFormat;
import java.util.*;

public class POSSystem {
    private static final String EMPLOYEE_DATABASE =
"Database/employeeDatabase.txt";
    private static final String EMPLOYEE_LOG_FILE =
"Database/employeeLogfile.txt";
    private static final DateFormat DATE_FORMAT = new SimpleDateFormat("yyyy-MM-
dd HH:mm:ss.SSS");

    private List<Employee> employees = new ArrayList<>();
    private String username = "";
    private String password = "";
    private String name = "";
    private Calendar calendar = null;
}

```

```

public void loadEmployeeData() {
    employees = readEmployeeDatabase();
}

private List<Employee> readEmployeeDatabase() {
    List<Employee> employeeList = new ArrayList<>();
    try (BufferedReader bufferedReader = new BufferedReader(new
FileReader(EMPLOYEE_DATABASE))) {
        String line;
        while ((line = bufferedReader.readLine()) != null) {
            Employee employee = parseEmployee(line);
            if (employee != null) {
                employeeList.add(employee);
            }
        }
    } catch (FileNotFoundException e) {
        System.err.println("Employee database not found: " +
EMPLOYEE_DATABASE);
    } catch (IOException e) {
        System.err.println("Error reading employee database: " +
EMPLOYEE_DATABASE);
    }
    return employeeList;
}

private Employee parseEmployee(String line) {
    try {
        String[] employeeDetails = line.split(" ");
        String name = employeeDetails[2] + " " + employeeDetails[3];
        return new Employee(employeeDetails[0], name, employeeDetails[1],
employeeDetails[4]);
    } catch (Exception e) {
        System.err.println("Error parsing employee line: " + line);
        return null;
    }
}

private void logActionToFile(String action, String username, String name,
String position, Calendar calendar) {
    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(EMPLOYEE_LOG_FILE, true))) {
        String log = String.format(
            "%s (%s %s) %s POS System. Time: %s",
            name, username, position, action,
            DATE_FORMAT.format(calendar.getTime()))

```

```

        );
        writer.write(log);
        writer.newLine();
    } catch (IOException e) {
        System.err.println("Error writing to employee log file: " +
EMPLOYEE_LOG_FILE);
    }
}

public int logIn(String userAuth, String passAuth) {
    loadEmployeeData();
    username = userAuth;
    calendar = Calendar.getInstance();

    Optional<Employee> matchingEmployee = employees.stream()
        .filter(employee -> employee.getUsername().equals(userAuth))
        .findFirst();

    if (matchingEmployee.isPresent()) {
        Employee employee = matchingEmployee.get();
        if (employee.getPassword().equals(passAuth)) {
            name = employee.getName();
            logActionToFile("logs into", username, name,
employee.getPosition(), calendar);

            if((employee.getPosition().equals("Cashier"))){
                return 1; //returns cashier status
            }
            else if((employee.getPosition().equals("Admin"))){
                return 2; //returns admin status
            }
        } else {
            System.out.println("Invalid password for username: " + username);
        }
    } else {
        System.out.println("Username not found: " + username);
    }
    return 0;
}

public void logOut(String position) {
    calendar = Calendar.getInstance();
    logActionToFile("logs out of", username, name, position, calendar);
}

```

```

public boolean checkTemp() {
    // TODO Auto-generated method stub
    return false;
}

public String continueFromTemp(long phoneNum) {
    // TODO Auto-generated method stub
    return null;
}

public void readFile() {
    // TODO Auto-generated method stub
}
}

```

3. POR class

Based on the checklist analysis, the **endPOS method** and **file handling code** in **deleteTempItem** and **retrieveTemp methods** were chosen for refactoring. These areas have opportunities for improvement in readability, maintainability, and performance.

Changes Made:

1. endPOS Method:

- **Issue:** Performs multiple tasks such as tax calculation, inventory updates, and file deletions. It also contains redundant logic.
- **Fix:** Split the endPOS method into smaller, single-purpose methods for tax calculation, inventory update, and cleanup.

2. File Handling Code:

- **Issue:** Duplicate file I/O logic in deleteTempItem and retrieveTemp. Manual resource management using standard try-catch-finally.
- **Fix:** Refactor file handling into reusable helper methods and replace standard try-catch with **try-with-resources** for automatic resource management.

3. Hardcoded Values:

- **Issue:** Hardcoded paths like "Database/newTemp.txt" make the code less flexible.
- **Fix:** Define constants for file paths.

Impact Analysis

1. Performance:

- **Before:** Redundant file handling logic, manual resource management.

- **After:** Reusable helper methods and efficient resource handling with try-with-resources.

2. Maintainability:

- **Before:** Long methods (endPOS) and hard-to-follow file operations.
- **After:** Smaller, single-purpose methods improve modularity and readability.

3. Readability:

- **Before:** Minimal comments, inconsistent naming, and hardcoded paths.
- **After:** Clear method names, constants for file paths, and meaningful comments.

4. Error Handling:

- **Before:** Generic error messages with System.out.println.
- **After:** Enhanced error logging with System.err.

This refactoring enhances both the **efficiency** and **readability** of the code, aligning it better with industry best practices.

Before Code

```
import java.io.*;

public class POR extends PointOfSale {
    long phoneNum;

    public POR(long phoneNum){
        this.phoneNum = phoneNum;
    };
    public void deleteTempItem(int id){
        try{
            String temp = "Database/newTemp.txt";
            if(System.getProperty("os.name").startsWith("W") || System.getProperty("os.name").startsWith("w")){
                //temp = "..\\Database\\newTemp.txt";
            }
            File tempF = new File(temp);
            FileReader fileR = new FileReader(tempFile);
            BufferedReader reader = new BufferedReader(fileR);
            BufferedWriter writer = new BufferedWriter(new FileWriter(tempF));
            String type= reader.readLine();
            String phone=reader.readLine();
            //this.phoneNum = Long.parseLong(phone);
            //System.out.println(this.phoneNum);
            writer.write(type);
        }
    }
}
```

```

        writer.write(System.getProperty("line.separator"));
        writer.write(phone);
        writer.write(System.getProperty("line.separator"));
        for (int i =0; i<transactionItem.size();i++){
            if (transactionItem.get(i).getItemID()!=id){
                writer.write(transactionItem.get(i).getItemID() +" "+
transactionItem.get(i).getAmount());
                writer.write(System.getProperty( "line.separator" ));
            }

        }
        fileR.close();
        writer.close();
        reader.close();
        File file = new File(tempFile);
        file.delete();
        tempF.renameTo(new File(tempFile));

    }
    catch(FileNotFoundException ex) {
        System.out.println(
            "Unable to open file 'temp'");
    }
    catch(IOException ex) {
        System.out.println(
            "Error reading file 'temp'");
    }
}

@SuppressWarnings("static-access")
public double endPOS(String textFile){
    Management man = new Management();
    man.addRental(this.phoneNum, this.transactionItem);
    detectSystem();
    if (transactionItem.size()>0){
        totalPrice = totalPrice*tax; //calculates price with tax
        //prints total with taxes
        //bool=payment();
        //f(bool==true){
            /*for (int counter = 0; counter < transactionItem.size(); counter++){
                //prints item name - price
                System.out.format("%d %s x %d --- $ %.2f\n",
transactionItem.get(counter).getItemID(),transactionItem.get(counter).getItemName
()),

```

```

        transactionItem.get(counter).getAmount(),
        transactionItem.get(counter).getPrice()*transactionItem.get(counter).getAmount());
    }
    System.out.format("Total with taxes: %.2f\n", totalPrice);
    inventory.updateInventory(textFile, transactionItem, databaseItem,true);
}*/
inventory.updateInventory(textFile, transactionItem, databaseItem,true);
}
//delete log file
File file = new File(tempFile);
file.delete();
databaseItem.clear();
transactionItem.clear();
return totalPrice;
}

public void retrieveTemp(String textFile){
    try{
        FileReader fileR = new FileReader(tempFile);
        BufferedReader textReader = new BufferedReader(fileR);
        String line=null;
        String[] lineSort;
        line=textReader.readLine();
        inventory.accessInventory(textFile, databaseItem);
        line=textReader.readLine();
        System.out.println("Phone number:");
        System.out.println(line);

        while ((line = textReader.readLine()) != null)
        {
            lineSort = line.split(" ");
            int itemNo = Integer.parseInt(lineSort[0]);
            int itemAmount = Integer.parseInt(lineSort[1]);
            enterItem(itemNo,itemAmount);
        }
        textReader.close();
        updateTotal();
    }
    catch(FileNotFoundException ex) {
        System.out.println(
            "Unable to open file 'temp'");
    }
    catch(IOException ex) {
        System.out.println(

```

```

        "Error reading file 'temp'");
    }
}
}

```

After Code

```

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class POR extends PointOfSale {
    private static final String TEMP_FILE_PATH = "Database/newTemp.txt";
    long phoneNum;

    public POR(long phoneNum) {
        this.phoneNum = phoneNum;
    }

    public void deleteTempItem(int id) {
        try {
            File tempFile = new File(TEMP_FILE_PATH + ".tmp");
            try (BufferedReader reader = new BufferedReader(new
            FileReader(TEMP_FILE_PATH));
                BufferedWriter writer = new BufferedWriter(new
            FileWriter(tempFile))) {

                // Write header information
                writer.write(reader.readLine() + System.lineSeparator()); // type
                writer.write(reader.readLine() + System.lineSeparator()); //
phone number

                // Write all items except the one to delete
                for (int i = 0; i < transactionItem.size(); i++) {
                    if (transactionItem.get(i).getItemID() != id) {
                        writer.write(transactionItem.get(i).getItemID() + " " +
transactionItem.get(i).getAmount() +
System.lineSeparator());
                    }
                }
            }

            // Replace original file with the updated file

```

```

        new File(TEMP_FILE_PATH).delete();
        tempFile.renameTo(new File(TEMP_FILE_PATH));

    } catch (IOException ex) {
        System.err.println("Error handling file: " + ex.getMessage());
    }
}

public double endPOS(String textFile) {
    if (!transactionItem.isEmpty()) {
        double totalWithTax = calculateTotalWithTax();
        updateInventoryAndRentalStatus(textFile);

        // Cleanup
        deleteTemporaryFile();
        clearData();
        return totalWithTax;
    }
    return 0;
}

private double calculateTotalWithTax() {
    return totalPrice * tax;
}

private void updateInventoryAndRentalStatus(String textFile) {
    inventory.updateInventory(textFile, transactionItem, databaseItem, true);
    Management management = new Management();
    management.addRental(phoneNum, transactionItem);
}

private void deleteTemporaryFile() {
    new File(TEMP_FILE_PATH).delete();
}

private void clearData() {
    transactionItem.clear();
    databaseItem.clear();
}

public void retrieveTemp(String textFile) {
    try (BufferedReader reader = new BufferedReader(new
FileReader(TEMP_FILE_PATH))) {
        inventory.accessInventory(textFile, databaseItem);
    }
}

```

```
System.out.println("Phone number: " + reader.readLine());
String line;
while ((line = reader.readLine()) != null) {
    String[] parts = line.split(" ");
    int itemNo = Integer.parseInt(parts[0]);
    int itemAmount = Integer.parseInt(parts[1]);
    enterItem(itemNo, itemAmount);
}

    updateTotal();
} catch (IOException ex) {
    System.err.println("Error reading temp file: " + ex.getMessage());
}
}
```